ASSESSMENT TOOL 3

ITA0403 – STATISTICS WITH R PROGRAMMING

Name: Sameer Ahmed G

Register Number: 192421154

Department: B Tech IT

Topic 1: Creating Matrices and Operations on Matrices

Question 1 (Level 3 – Apply)

Question focus: Row-wise sales matrix; apply() totals and averages

Aim: Create a labeled 3 × 4 sales matrix and calculate product totals and quarterly averages.

Algorithm:

1. Identify the required R object and its dimensions or class.
2. Apply the relevant R function(s) or operators.
3. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

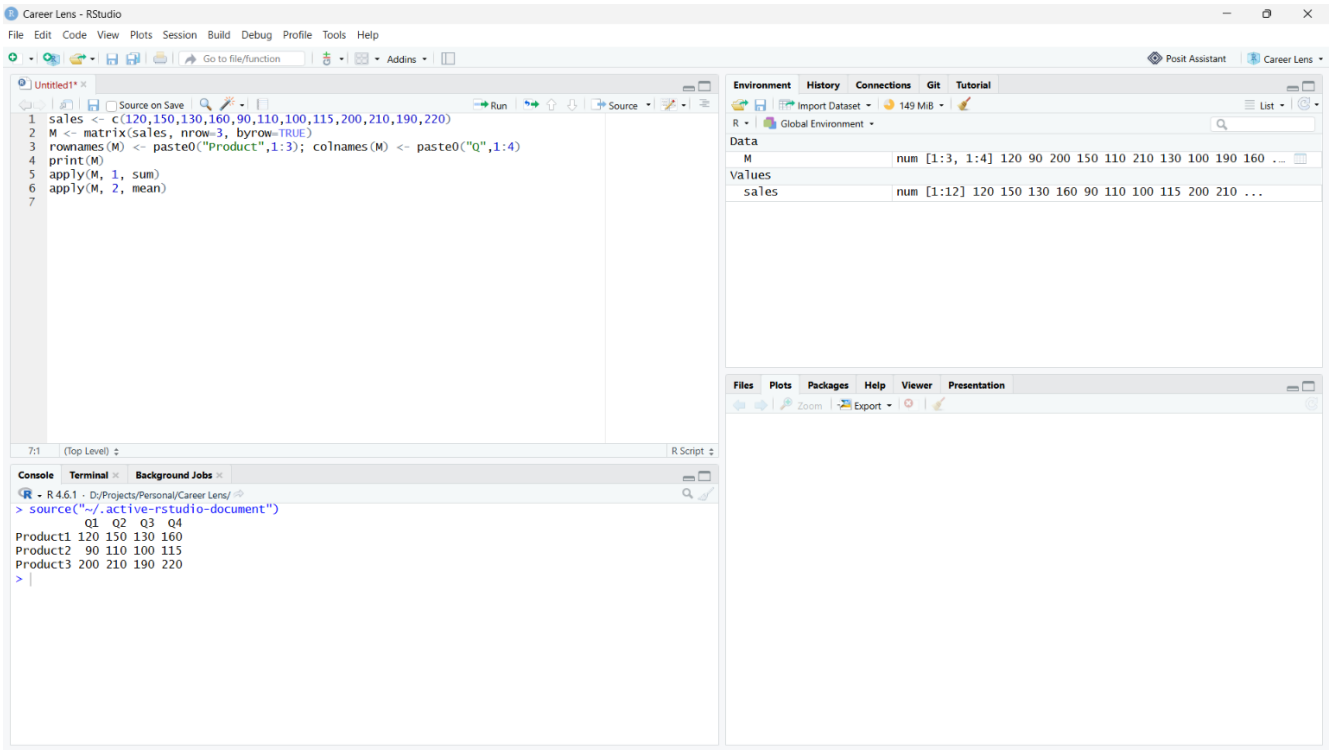
R Program:

```
sales <- c(120,150,130,160,90,110,100,115,200,210,190,220)
M <- matrix(sales, nrow=3, byrow=TRUE)
rownames(M) <- paste0("Product",1:3); colnames(M) <- paste0("Q",1:4)
print(M)
apply(M, 1, sum)
apply(M, 2, mean)
```

Expected Output / Analysis:

```
         Q1  Q2  Q3  Q4
Product1 120 150 130 160
Product2  90 110 100 115
Product3 200 210 190 220
Row totals: 560 415 820
Quarter means: 136.67 156.67 140.00 165.00
```

Result: MARGIN=1 applies the function to rows; MARGIN=2 applies it to columns. The product totals are 560, 415 and 820.



Question 2 (Level 4 – Analyze)

Question focus: Element-wise versus matrix multiplication

Aim: Explain the dimension rules for * and %*% and make element-wise multiplication compatible.

Algorithm:

4. Identify the required R object and its dimensions or class.
5. Apply the relevant R function(s) or operators.
6. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

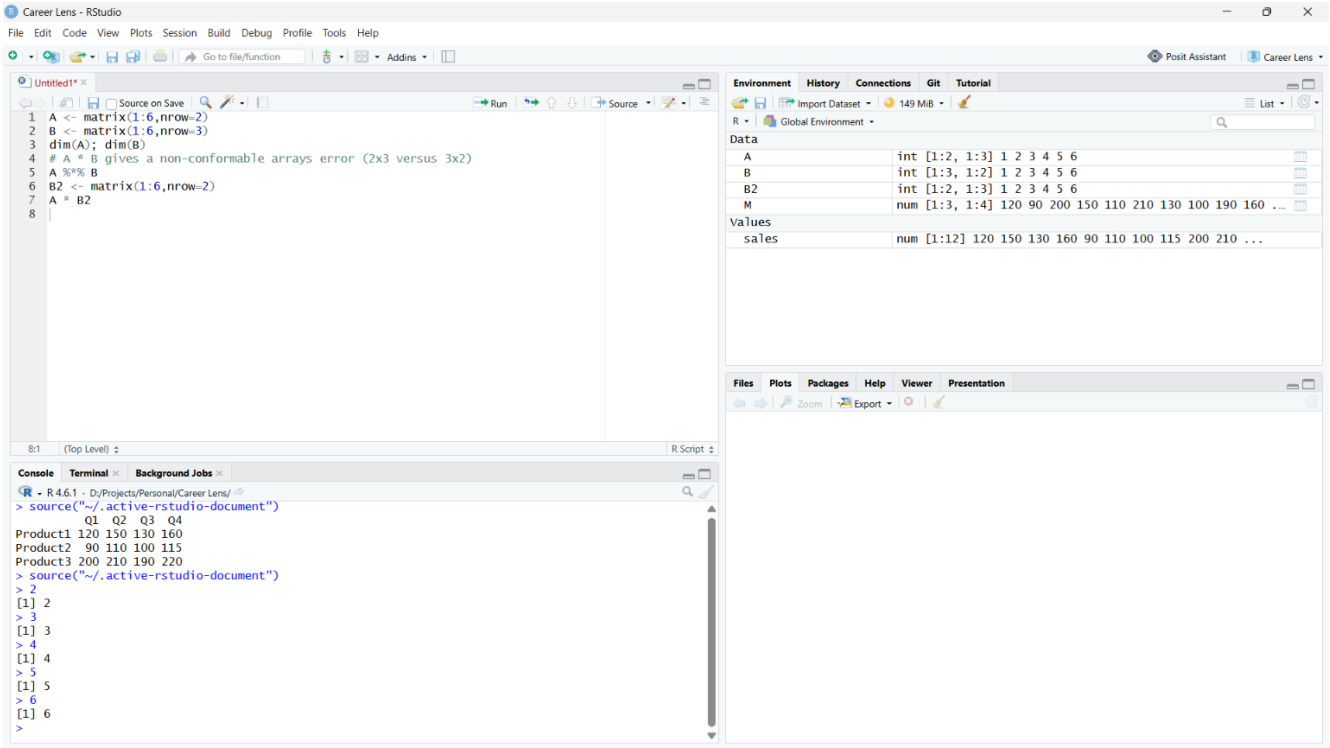
R Program:

```
A <- matrix(1:6,nrow=2)
B <- matrix(1:6,nrow=3)
dim(A); dim(B)
# A * B gives a non-conformable arrays error (2x3 versus 3x2)
A %*% B
B2 <- matrix(1:6,nrow=2)
A * B2
```

Expected Output / Analysis:

```
dim(A) = 2 3; dim(B) = 3 2
A %*% B =       [,1] [,2]
[1,]    22   49
[2,]    28   64
A * B2 =       [,1] [,2] [,3]
[1,]     1    9   25
[2,]     4   16   36
```

Result: For %*%, ncol(A) must equal nrow(B); the result is 2 × 2. For *, the matrices must have compatible dimensions; making B2 2 × 3 allows element-wise multiplication.



Question 3 (Level 3 – Apply)

Question focus: Transpose, determinant and inverse

Aim: Use t(), det() and solve() and verify the inverse.

Algorithm:

7. Identify the required R object and its dimensions or class.
8. Apply the relevant R function(s) or operators.
9. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

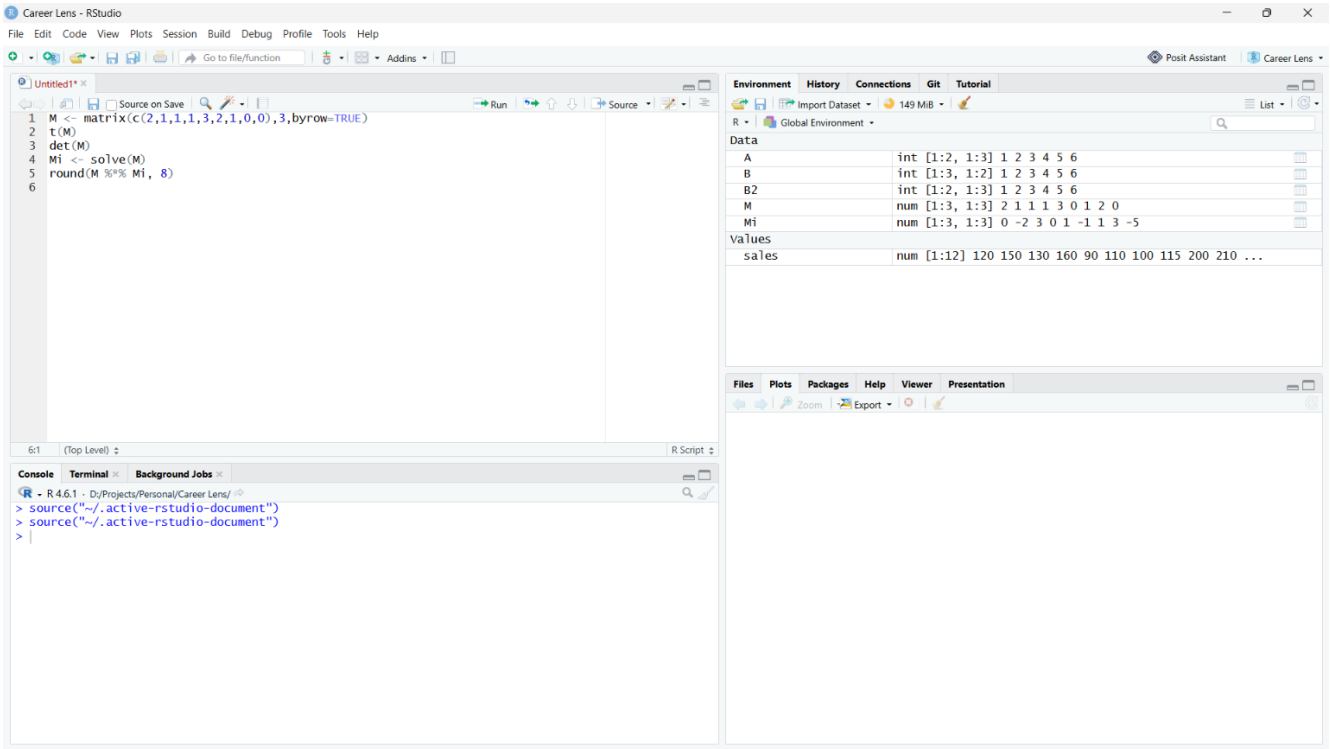
R Program:

```
M <- matrix(c(2,1,1,1,3,2,1,0,0),3,byrow=TRUE)
t(M)
det(M)
Mi <- solve(M)
round(M %*% Mi, 8)
```

Expected Output / Analysis:

```
det(M) = -1
round(M %*% solve(M), 8) =
     [,1] [,2] [,3]
[1,]    1    0    0
[2,]    0    1    0
[3,]    0    0    1
```

Result: The non-zero determinant means M is invertible. round() removes tiny floating-point errors when displaying the identity matrix.



Question 4 (Level 4 – Analyze)

Question focus: Linear system and singular matrix

Aim: Solve the system and explain why dependent rows make solve() fail.

Algorithm:

10. Identify the required R object and its dimensions or class.

11. Apply the relevant R function(s) or operators.

12. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

R Program:

```
A <- matrix(c(2,1,1, 1,3,2, 1,0,0),3,byrow=TRUE)
b <- c(7,13,1)
solve(A,b)
A2 <- A; A2[3,] <- c(2,6,4)
det(A2)
# solve(A2,b) fails: A2 is singular
```

Expected Output / Analysis:

```
Original solution: x=1, y=2, z=3
Modified matrix determinant: 0
solve(A2,b): singular-system error
```

Result: The modified third row is twice row 2, so rows are linearly dependent, rank is below 3 and determinant is zero.

RStudio Screenshot: [Paste screenshot of the code and console output here.]

Question 5 (Level 3 – Apply)

Question focus: Indexing and resizing matrices

Aim: Extract rows, columns and a submatrix; select values and append a row and column.

Algorithm:

13. Identify the required R object and its dimensions or class.

14. Apply the relevant R function(s) or operators.

15. Display the result and check it against the expected behaviour.

Pseudocode:

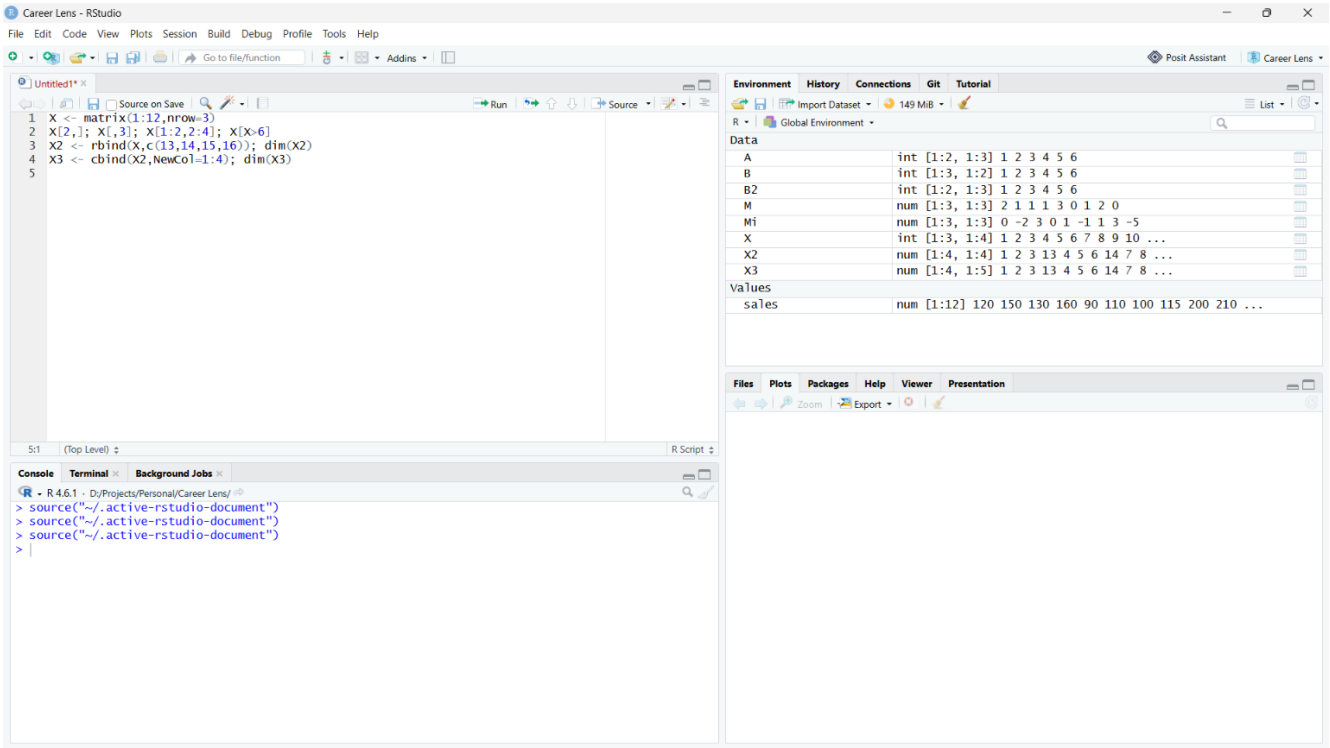
```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

R Program:

```
X <- matrix(1:12,nrow=3)
X[2,]; X[,3]; X[1:2,2:4]; X[X>6]
X2 <- rbind(X,c(13,14,15,16)); dim(X2)
X3 <- cbind(X2,NewCol=1:4); dim(X3)
```

Expected Output / Analysis:

```
Row 2: 2 5 8 11
Column 3: 7 8 9
Submatrix: rows 1:2, columns 2:4
Values > 6: 7 8 9 10 11 12
dim(X2) = 4 4; dim(X3) = 4 5
```



Question 6 (Level 4 – Analyze)

Question focus: rbind() recycling

Aim: Analyze the row-length mismatch and avoid unintended recycling.

Algorithm:

16. Identify the required R object and its dimensions or class.
17. Apply the relevant R function(s) or operators.
18. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

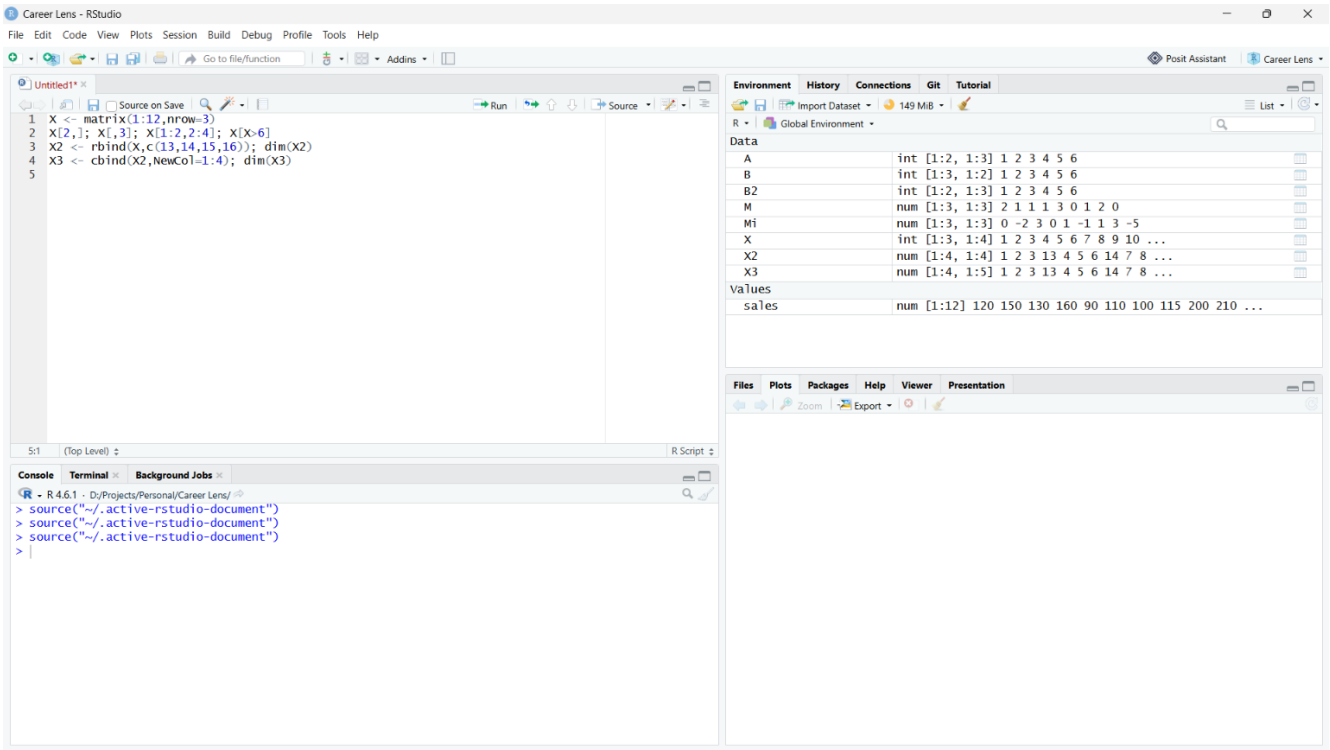
R Program:

```
m <- matrix(1:6,nrow=2)
rbind(m,7:8) # recycled to three columns; warning may be issued
rbind(m,7:9) # exact length
```

Expected Output / Analysis:

```
m has 3 columns. The short vector may be recycled to row 7 8 7, with a warning because 3 is not a multiple of 2. rbind(m,7:9) appends 7 8 9 without recycling.
```

Result: Check length(new_row) == ncol(m) before rbind(); the appended row should have exactly three elements.



Question 7 (Level 3 – Apply)

Question focus: Diagonal, trace and upper triangle

Aim: Extract diagonal values, sum them and zero elements below the diagonal.

Algorithm:

19. Identify the required R object and its dimensions or class.

20. Apply the relevant R function(s) or operators.

21. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

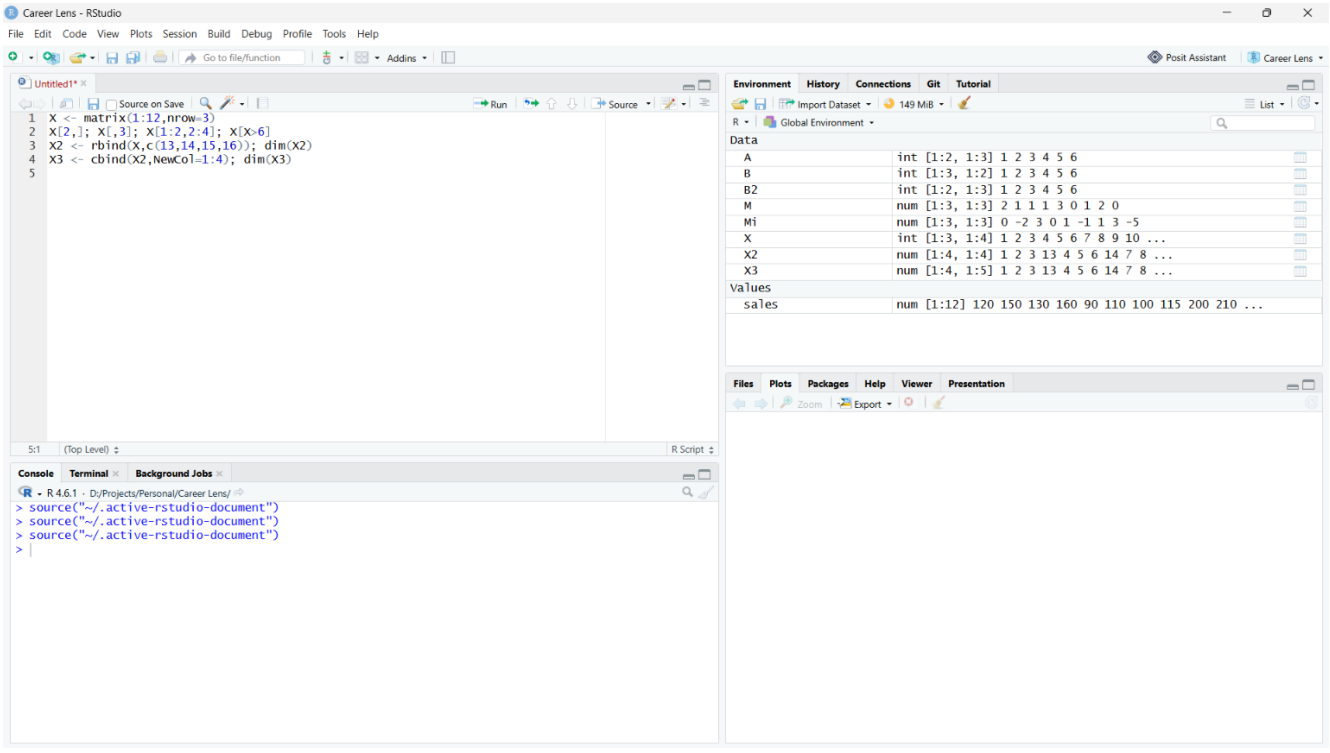
R Program:

```
M <- matrix(1:16,nrow=4)
diag(M); sum(diag(M))
M[upper.tri(M)]
M[lower.tri(M)] <- 0
M
```

Expected Output / Analysis:

```
Diagonal: 1 6 11 16
Trace: 34
Above diagonal: 5 9 13 10 14 15
Modified M is upper triangular, with zeros below the main diagonal.
```

Result: upper.tri() returns a logical matrix that is TRUE above the main diagonal and can be used for extraction or assignment.



Question 8 (Level 4 – Analyze)

Question focus: apply(), rowSums() and colSums()

Aim: Compare row and column sums and demonstrate a custom apply() function.

Algorithm:

22. Identify the required R object and its dimensions or class.
23. Apply the relevant R function(s) or operators.
24. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

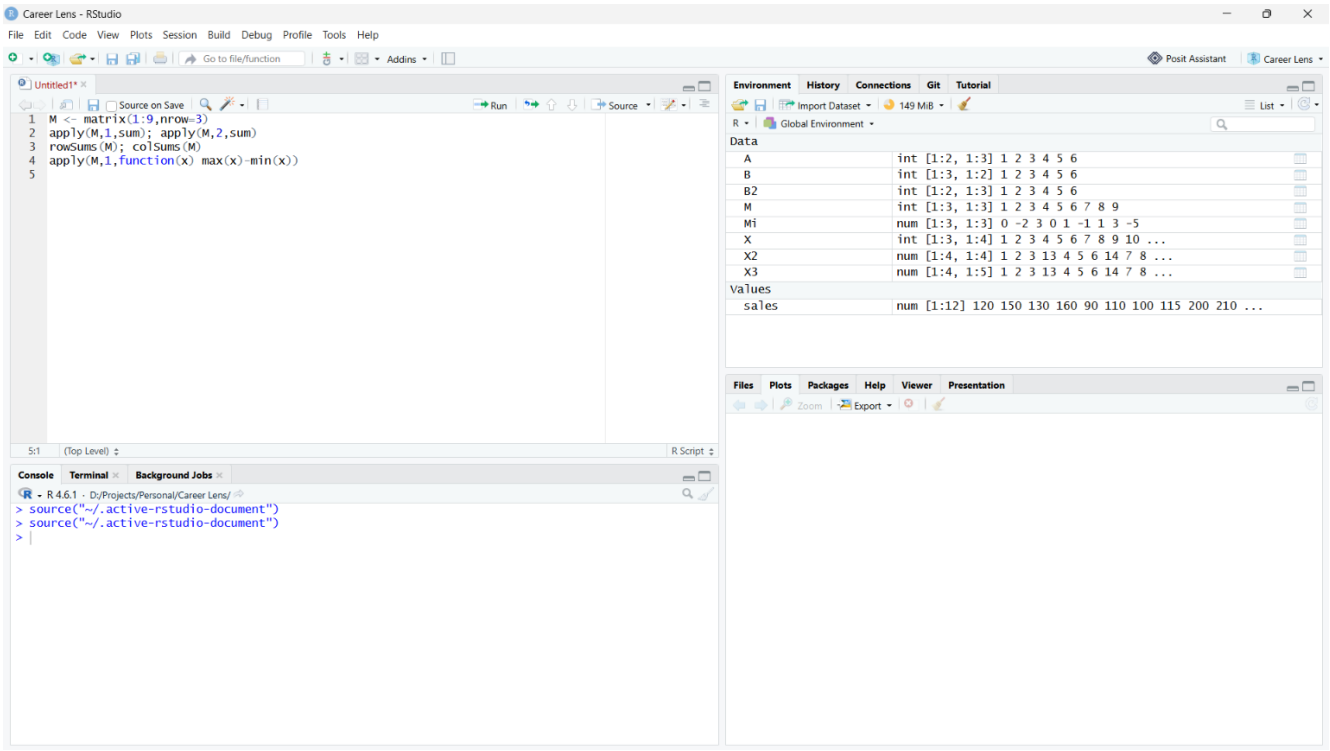
R Program:

```
M <- matrix(1:9,nrow=3)
apply(M,1,sum); apply(M,2,sum)
rowSums(M); colSums(M)
apply(M,1,function(x) max(x)-min(x))
```

Expected Output / Analysis:

```
Row sums: 12 15 18
Column sums: 6 15 24
rowSums/colSums return the same respective vectors.
Row ranges: 6 6 6
```

Result: MARGIN=1 works across rows; MARGIN=2 works across columns. apply() is useful for custom summaries such as max-min that rowSums() does not calculate.



Question 9 (Level 3 – Apply)

Question focus: Totals, percentages and class averages

Aim: Add total and percentage columns, find qualifying students and compute subject means.

Algorithm:

25. Identify the required R object and its dimensions or class.
26. Apply the relevant R function(s) or operators.
27. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

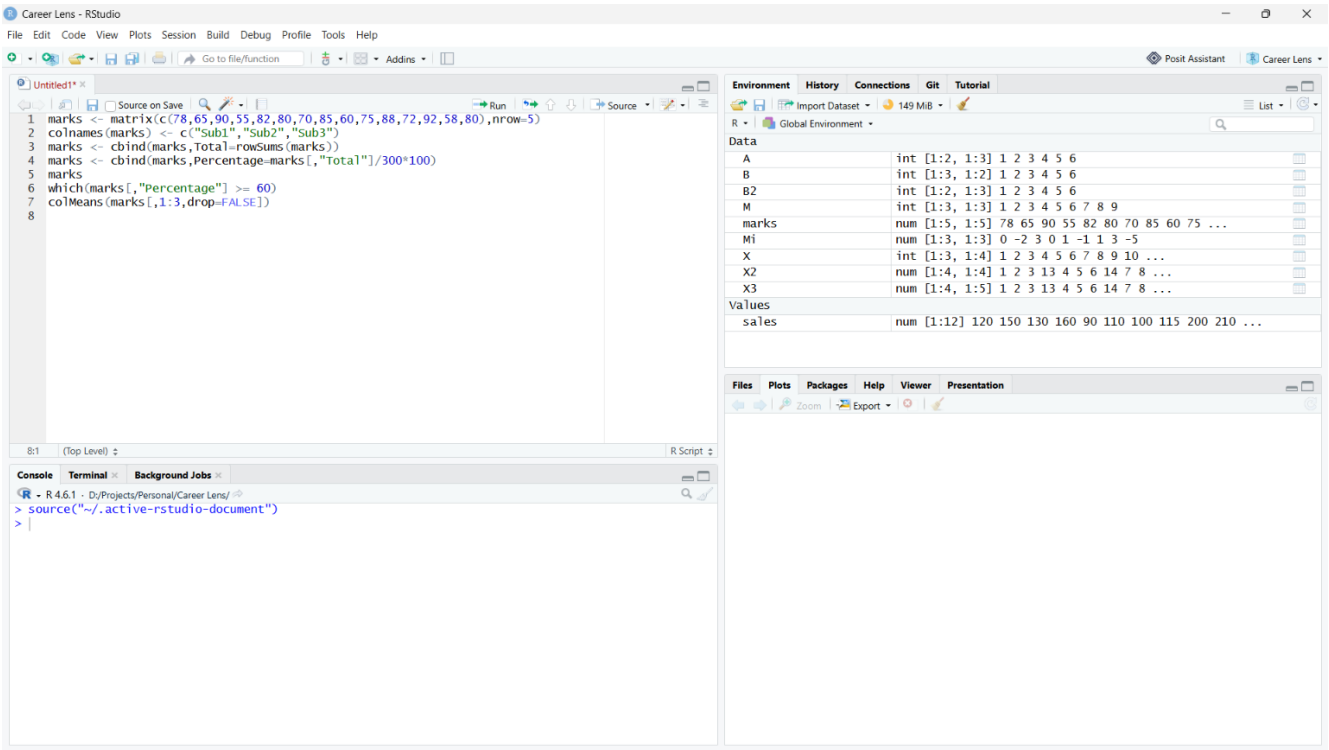
R Program:

```
marks <- matrix(c(78,65,90,55,82,80,70,85,60,75,88,72,92,58,80),nrow=5)
colnames(marks) <- c("Sub1","Sub2","Sub3")
marks <- cbind(marks,Total=rowSums(marks))
marks <- cbind(marks,Percentage=marks[,"Total"]/300*100)
marks
which(marks[,"Percentage"] >= 60)
colMeans(marks[,1:3,drop=FALSE])
```

Expected Output / Analysis:

```
Creates a 5 × 5 matrix with Sub1, Sub2, Sub3, Total and Percentage columns. which()
returns row indices with percentage >= 60; colMeans() returns three subject means.
```

Result: The question does not supply actual marks, so the program uses sample values. Replace them with the class marks if different.



Question 10 (Level 4 – Analyze)

Question focus: Type coercion in cbind()

Aim: Explain why adding a character Result column changes a numeric matrix and use a data frame instead.

Algorithm:

28. Identify the required R object and its dimensions or class.
29. Apply the relevant R function(s) or operators.
30. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

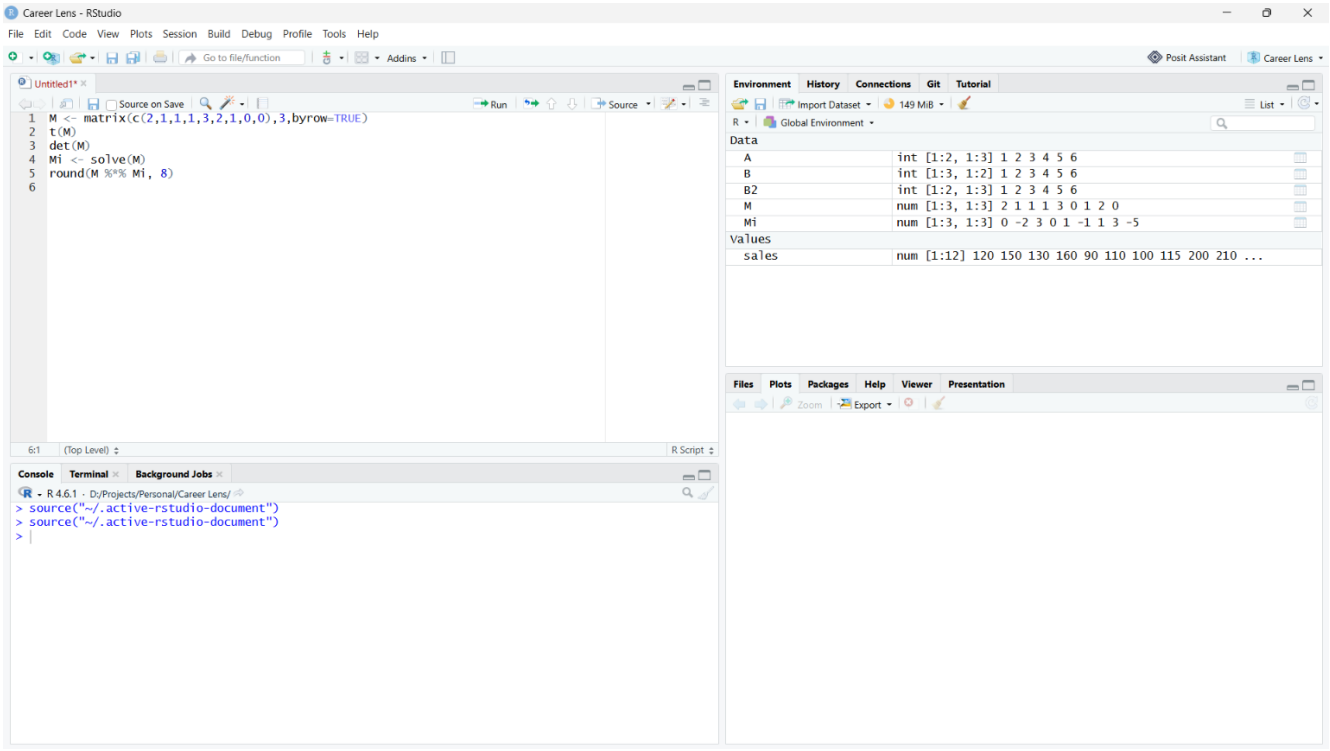
R Program:

```
marks <- matrix(c(70,80,55,90,60,75,85,50,88,65,70,92,45,78,80),nrow=5)
result <- c("Pass","Fail","Pass","Pass","Fail")
combined <- cbind(marks,Result=result)
typeof(combined) # "character"
# rowSums(combined) fails: matrix values are character
df <- data.frame(marks,Result=result)
rowSums(df[,1:3])
```

Expected Output / Analysis:

```
typeof(combined) = "character"; numeric-looking values are coerced to character.
rowSums(combined) errors. In the data frame, the marks columns remain numeric and Result
remains character.
```

Result: An atomic matrix stores one underlying type. A data frame stores columns separately, making it suitable for mixed numeric and character data.



Topic 2: Creating Arrays and Operations on Arrays

Question 1 (Level 3 – Apply)

Question focus: Create and label a 3D array

Aim: Create a 3 × 4 × 2 array, name its dimensions and access an element by index and name.

Algorithm:

31. Identify the required R object and its dimensions or class.

32. Apply the relevant R function(s) or operators.

33. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

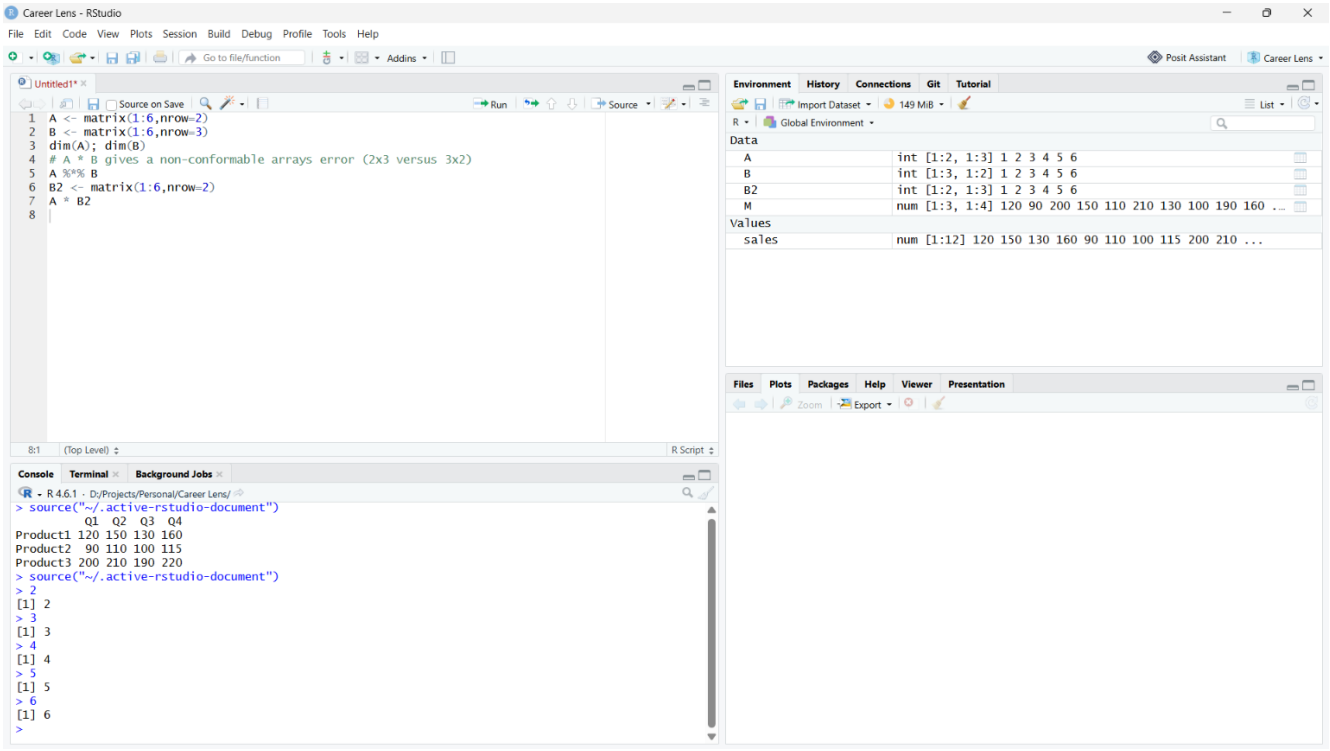
R Program:

```
a <- array(1:24,dim=c(3,4,2),
 dimnames=list(c("R1","R2","R3"),paste0("C",1:4),c("Sheet1","Sheet2")))
a; dim(a); length(a)
a[2,3,2]
a["R2","C3","Sheet2"]
```

Expected Output / Analysis:

```
dim(a): 3 4 2; length(a): 24
Both element lookups return 17.
```

Result: R arrays fill column-wise by default. Numeric and named indexing refer to the same element.



Question 2 (Level 3 – Apply)

Question focus: Aggregate branch, quarter and year sales

Aim: Use apply() with margins 1, 2 and 3.

Algorithm:

34. Identify the required R object and its dimensions or class.
35. Apply the relevant R function(s) or operators.
36. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

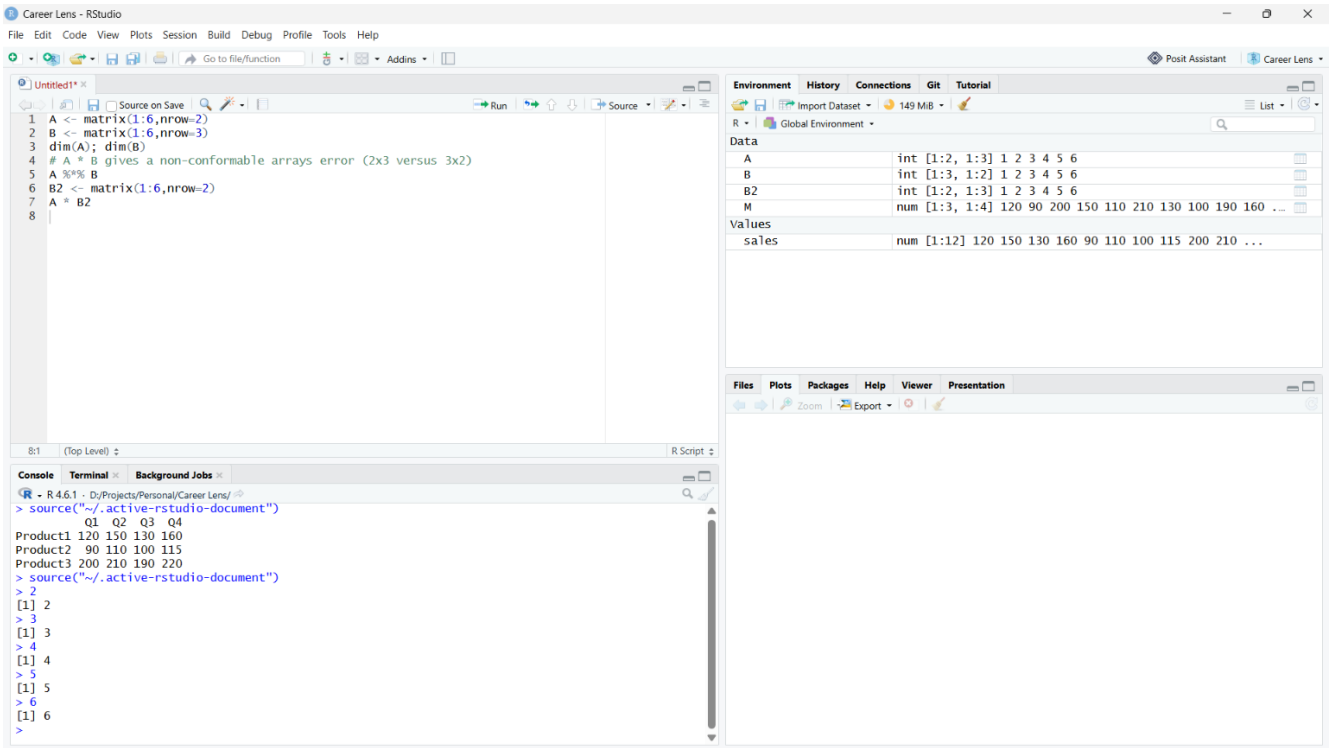
R Program:

```
sales <- array(1:24,dim=c(3,4,2))
apply(sales,1,sum) # branch totals
apply(sales,2,sum) # quarter totals
apply(sales,3,sum) # year totals
```

Expected Output / Analysis:

Branch totals: 92 100 108
Quarter totals: 48 66 84 102
Year totals: 78 222

Result: MARGIN identifies dimensions retained; all remaining dimensions are summed away.



Question 3 (Level 4 – Analyze)

Question focus: Dimension dropping

Aim: Determine output shapes for array indexing and preserve dimensions with drop=FALSE.

Algorithm:

37. Identify the required R object and its dimensions or class.
38. Apply the relevant R function(s) or operators.
39. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

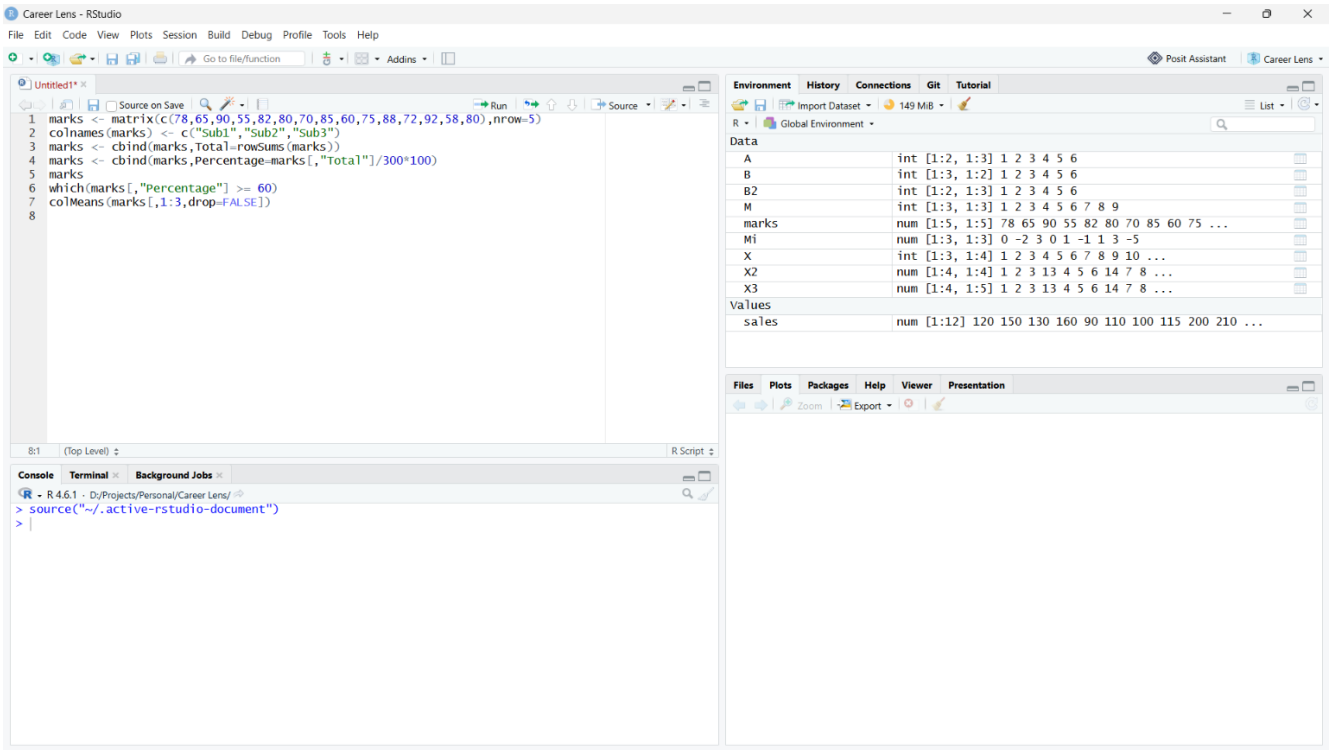
R Program:

```
a <- array(1:24,dim=c(2,3,4))
dim(a[1,,]); a[1,,]
dim(a[,2,3]); a[,2,3]
a[2,3,4]
dim(a[1,,,drop=FALSE])
```

Expected Output / Analysis:

```
a[1,,] is a 3 × 4 matrix; a[,2,3] is a length-2 vector; a[2,3,4] is scalar 24. With
drop=FALSE, dimensions are 1 × 3 × 4.
```

Result: R drops dimensions of length one by default. Preserve dimensions when downstream code requires a stable array shape.



Question 4 (Level 3 – Apply)

Question focus: Array arithmetic and comparisons

Aim: Compute sums, products, comparisons and positions in arrays.

Algorithm:

40. Identify the required R object and its dimensions or class.
41. Apply the relevant R function(s) or operators.
42. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

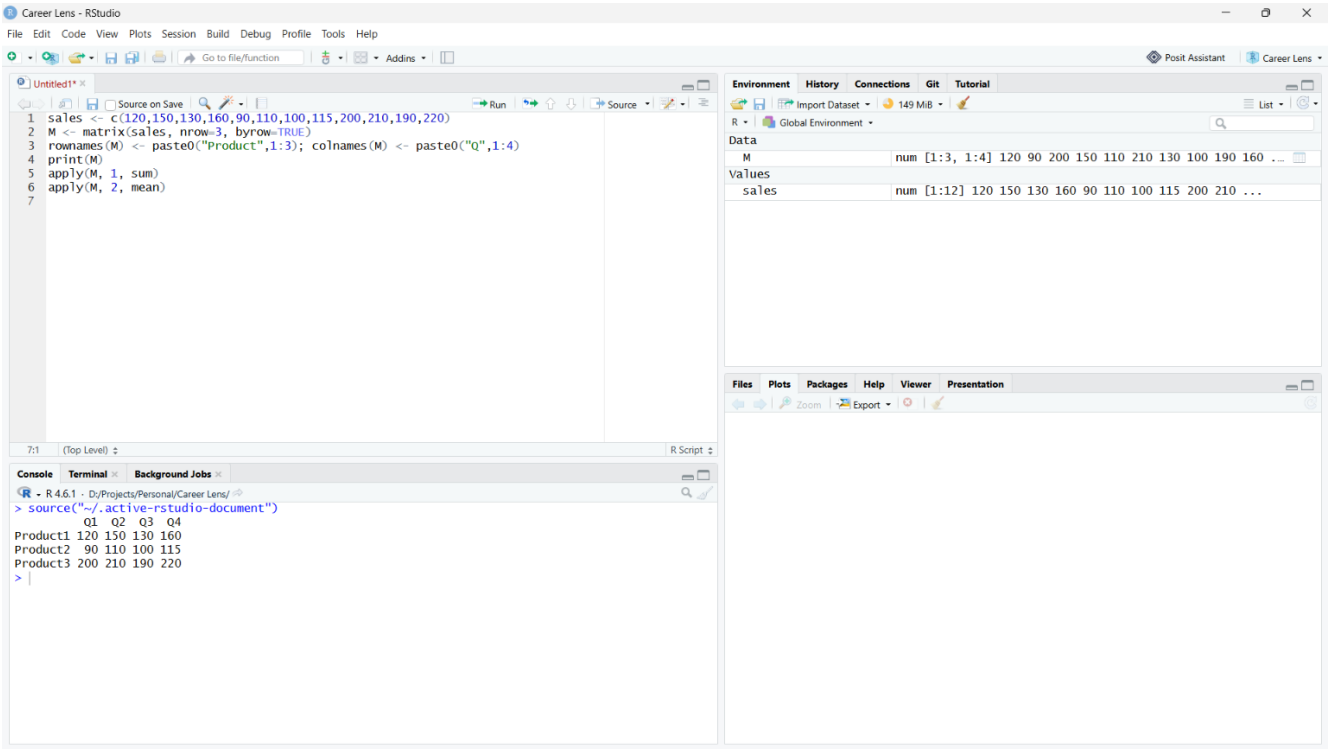
R Program:

```
a1 <- array(1:12,dim=c(2,3,2)); a2 <- array(12:1,dim=c(2,3,2))
a1+a2
a1*a2
a1>a2
sum(a1>a2)
which(a1==6)
```

Expected Output / Analysis:

```
a1+a2: every element is 13
sum(a1>a2): 6
which(a1==6): 6
The product is element-wise; inspect print(a1*a2) for the product array.
```

Result: Equal dimensions permit element-wise arithmetic. sum() counts TRUE values; which() gives linear positions in column-major order.



Question 5 (Level 4 – Analyze)

Question focus: Array recycling and truncation

Aim: Explain short input recycling and extra-value truncation.

Algorithm:

43. Identify the required R object and its dimensions or class.
44. Apply the relevant R function(s) or operators.
45. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

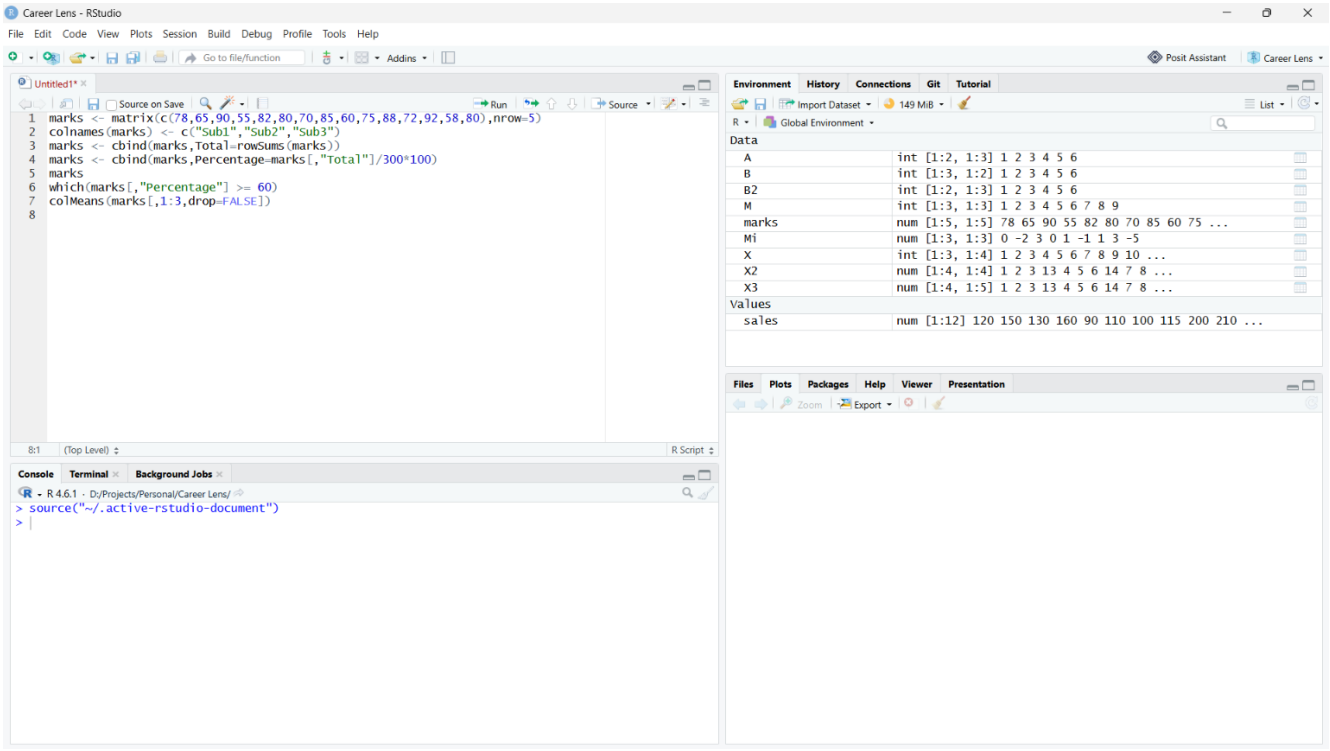
R Program:

```
short_arr <- array(1:5,dim=c(2,3,2))
short_arr; short_arr[,,2]
long_arr <- array(1:30,dim=c(2,3,2))
length(long_arr); long_arr
```

Expected Output / Analysis:

```
12 elements are required. 1:5 is recycled to fill the array and a warning occurs because
12 is not a multiple of 5. With 1:30, only the first 12 values are used;
length(long_arr)=12.
```

Result: Validate length(data) == prod(dimensions) when every source value must be used exactly once.



Question 6 (Level 3 – Apply)

Question focus: Permute dimensions using aperm()

Aim: Permute dimensions and verify the corresponding element position.

Algorithm:

46. Identify the required R object and its dimensions or class.
47. Apply the relevant R function(s) or operators.
48. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

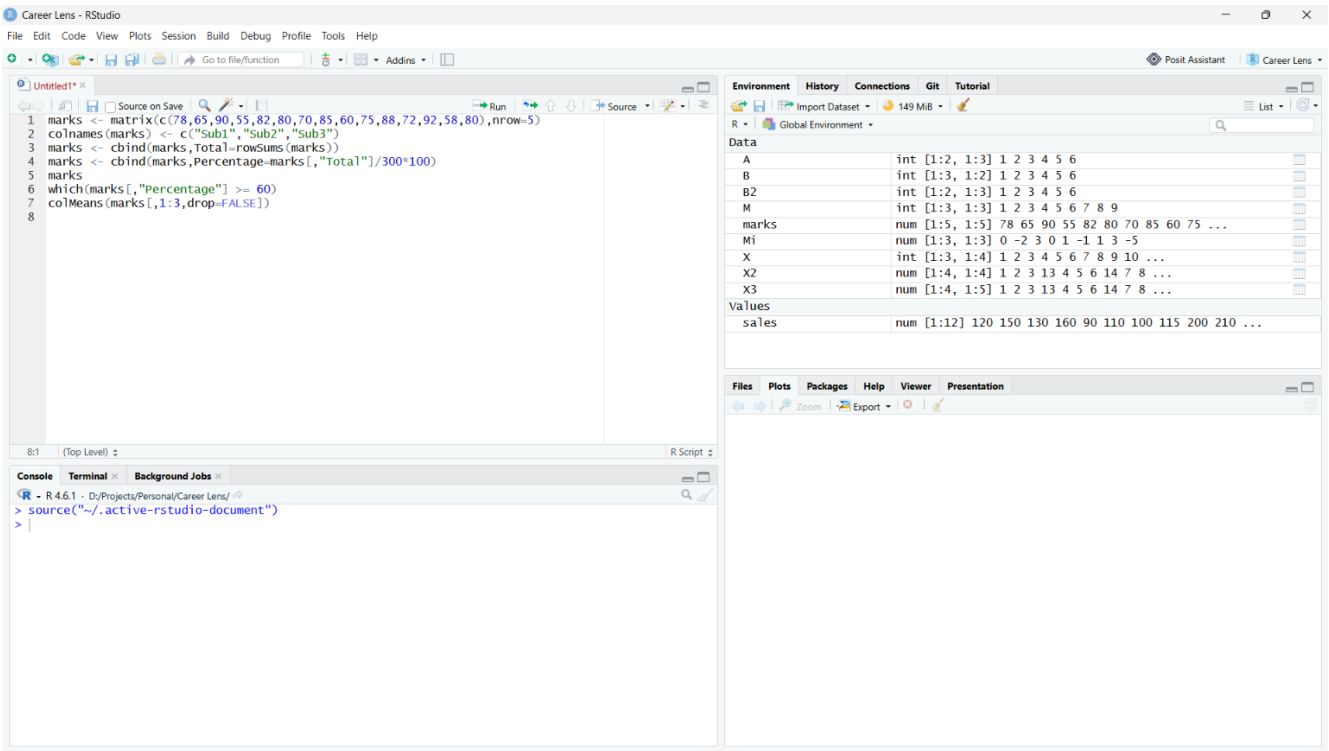
R Program:

```
a <- array(1:24,dim=c(2,3,4))
p1 <- aperm(a,c(2,1,3))
dim(p1)
a[1,2,3]; p1[2,1,3]
p2 <- aperm(a,c(3,1,2)); dim(p2)
```

Expected Output / Analysis:

```
dim(p1): 3 2 4
a[1,2,3] and p1[2,1,3] both equal 14.
dim(p2): 4 2 3.
```

Result: aperm() permutes any number of dimensions; t() transposes a two-dimensional matrix.



Question 7 (Level 4 – Analyze)

Question focus: Array ordering versus cbind/rbind

Aim: Compare combining matrices into an array, by columns and by rows.

Algorithm:

49. Identify the required R object and its dimensions or class.

50. Apply the relevant R function(s) or operators.

51. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

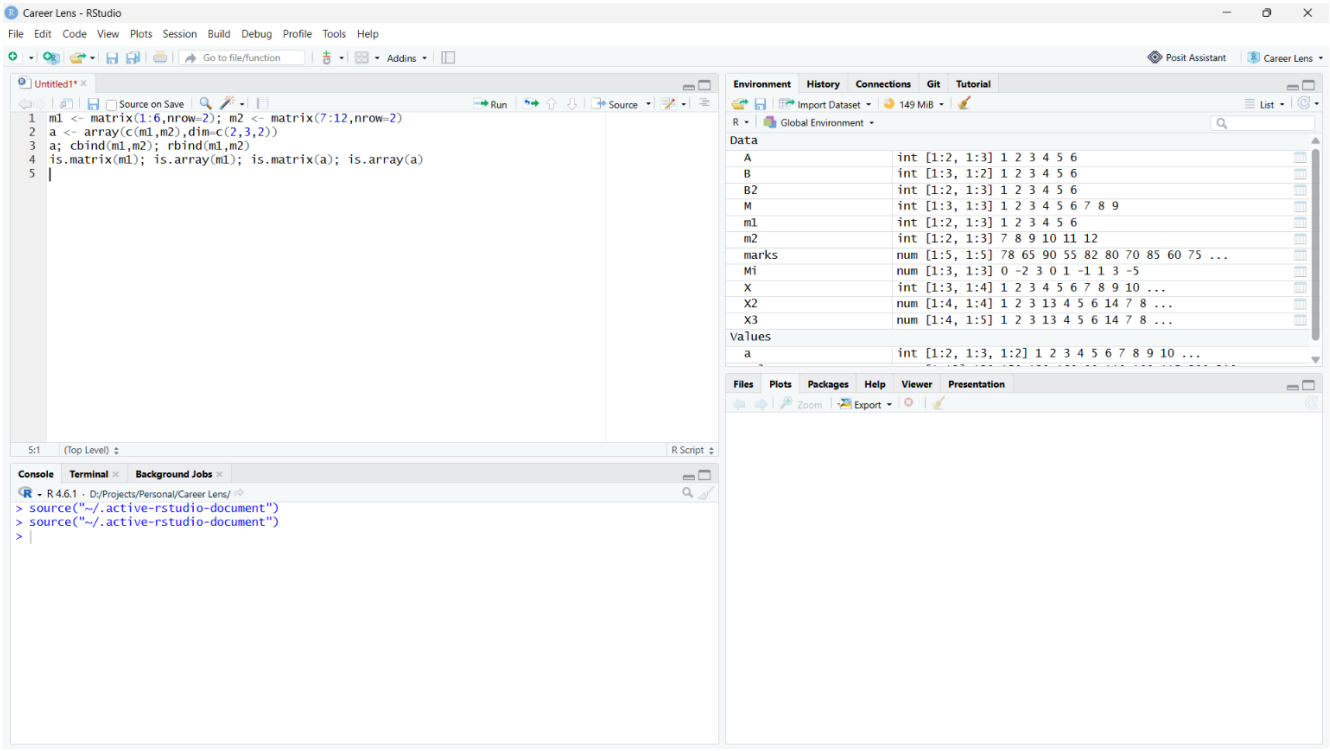
R Program:

```
m1 <- matrix(1:6,nrow=2); m2 <- matrix(7:12,nrow=2)
a <- array(c(m1,m2),dim=c(2,3,2))
a; cbind(m1,m2); rbind(m1,m2)
is.matrix(m1); is.array(m1); is.matrix(a); is.array(a)
```

Expected Output / Analysis:

```
The first layer of a is m1 and the second is m2. cbind() gives 2 × 6; rbind() gives 4 × 3. For m1: is.matrix and is.array are TRUE. For a: is.matrix is FALSE and is.array is TRUE.
```

Result: A matrix is a two-dimensional array. A three-dimensional array is not a matrix.



Question 8 (Level 3 – Apply)

Question focus: Temperature summaries

Aim: Calculate city means, seasonal maxima, maximum position and a city-season slice.

Algorithm:

52. Identify the required R object and its dimensions or class.
53. Apply the relevant R function(s) or operators.
54. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

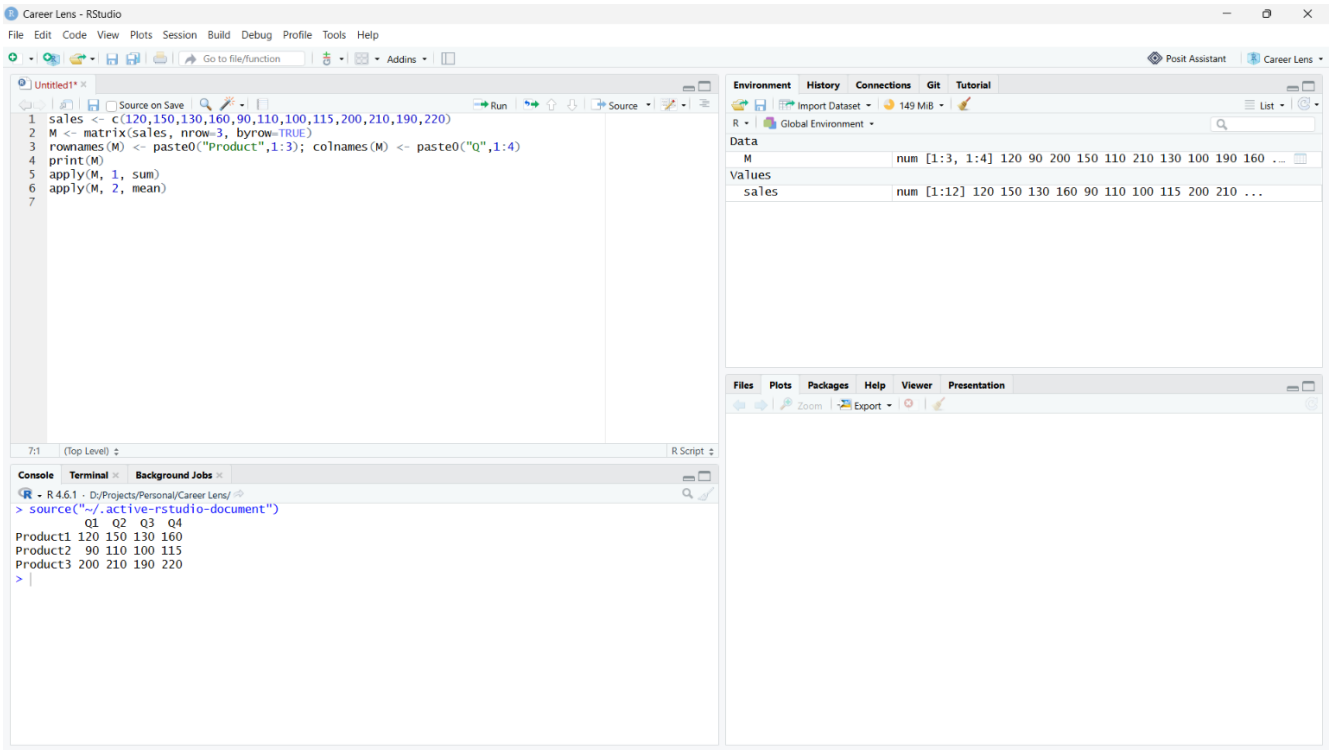
R Program:

```
# temp must be a numeric array with dimensions 3 x 7 x 2
apply(temp,1,mean)
apply(temp,3,max)
which.max(temp[,,1])
temp[2,,2]
# To get the day with the highest temperature across cities:
which.max(apply(temp[,,1],2,max))
```

Expected Output / Analysis:

```
Exact numeric values depend on the values stored in temp. The first call returns 3 city
means; the second returns 2 seasonal maxima; temp[2,,2] returns 7 temperatures.
which.max(temp[,,1]) is a linear position in the city-by-day slice, not directly a day
number.
```

Result: No temperature values were supplied, so exact numeric output cannot be calculated. Aggregate by day before using which.max() if a day number is required.



Question 9 (Level 4 – Analyze)

Question focus: MARGIN vectors in 3D apply()

Aim: Compare retained dimensions and predict results for different MARGIN values.

Algorithm:

55. Identify the required R object and its dimensions or class.
56. Apply the relevant R function(s) or operators.
57. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

R Program:

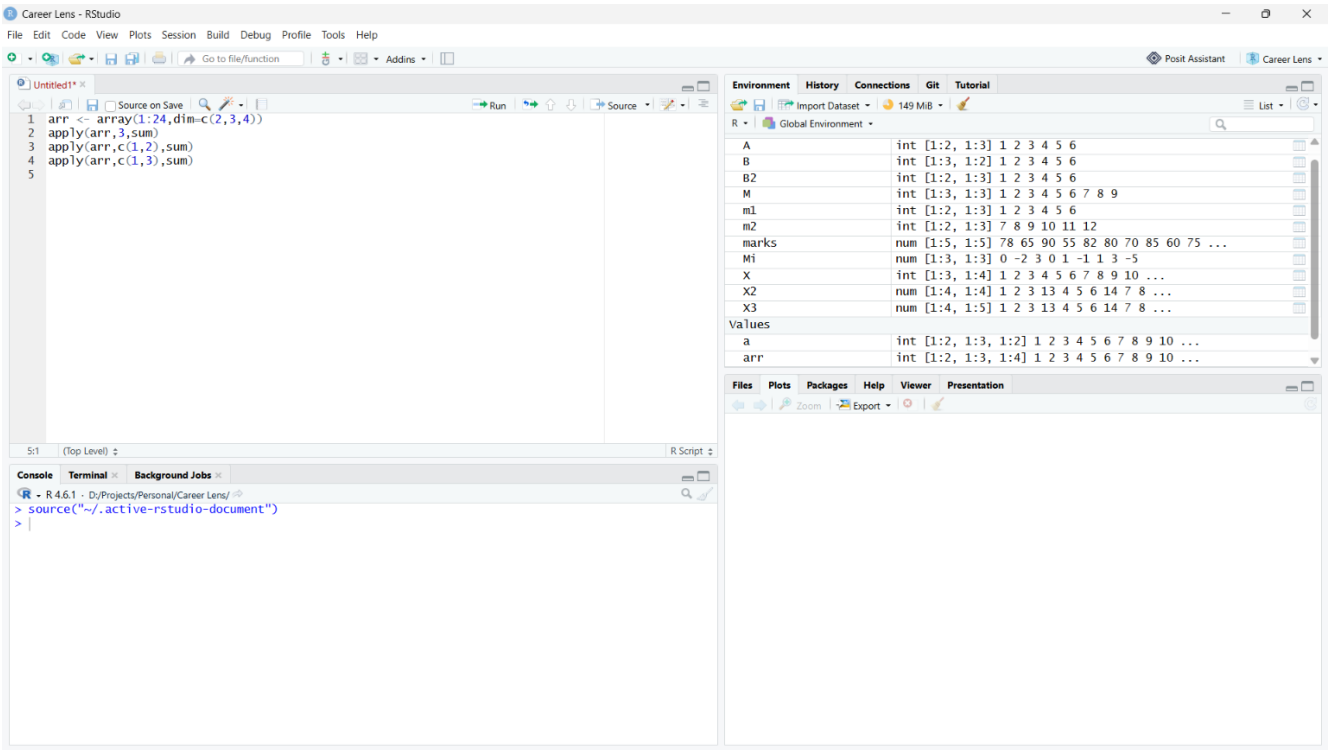
```
arr <- array(1:24,dim=c(2,3,4))
apply(arr,3,sum)
apply(arr,c(1,2),sum)
apply(arr,c(1,3),sum)
```

Expected Output / Analysis:

apply(arr,3,sum): 21 57 93 129
apply(arr,c(1,2),sum):

```
     [,1] [,2] [,3]
[1,]   40   48   56
[2,]   44   52   60
apply(arr,c(1,3),sum):
     [,1] [,2] [,3] [,4]
[1,]    9   27   45   63
[2,]   12   30   48   66
```

Result: MARGIN lists the dimensions retained. Dimensions not listed are collapsed by sum(); c(1,3) therefore returns a 2 × 4 matrix.



Question 10 (Level 3 – Apply)

Question focus: Replace negative values and scale a layer

Aim: Use logical indexing, multiply the second layer by 10 and compare sums.

Algorithm:

58. Identify the required R object and its dimensions or class.
59. Apply the relevant R function(s) or operators.
60. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

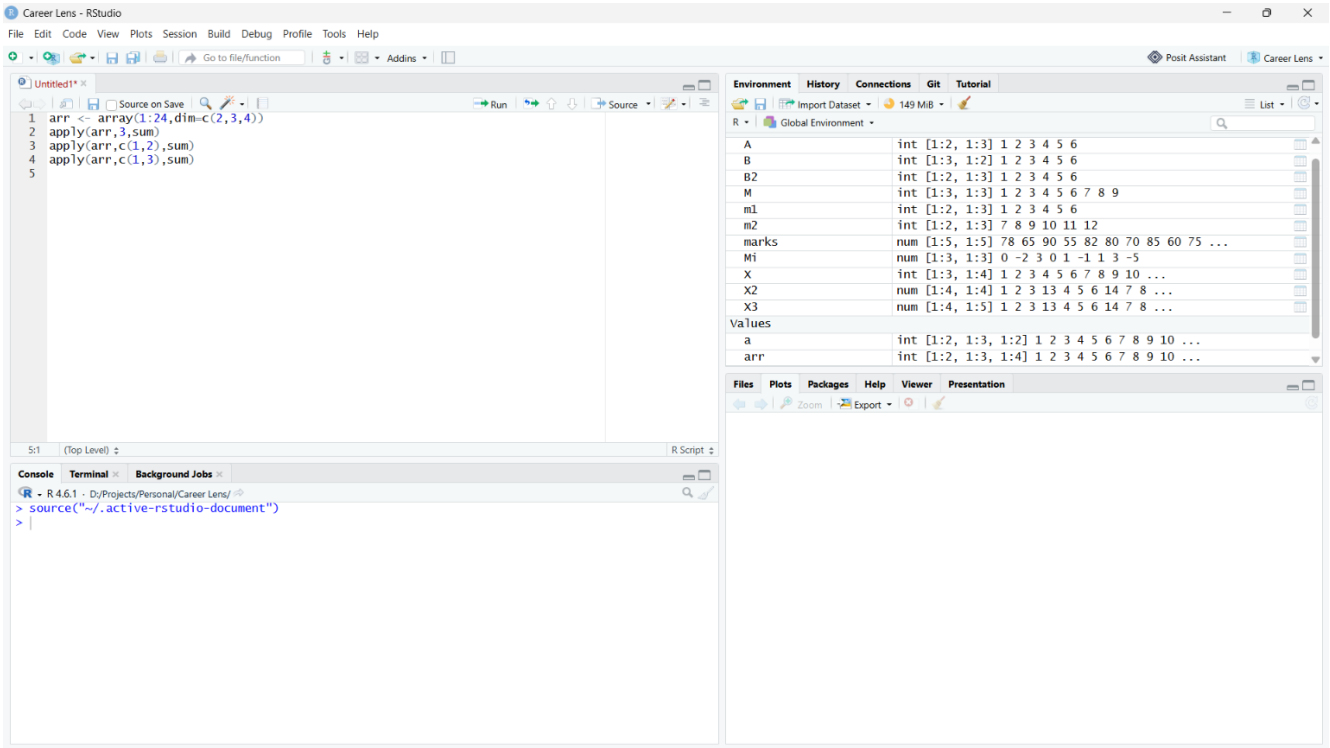
R Program:

```
arr <- array(c(-3,5,-1,8,2,-7,4,-6,9,-2,1,-5),dim=c(2,3,2))
original <- arr; before <- sum(arr)
arr[arr<0] <- 0
arr[,,2] <- arr[,,2]*10
after <- sum(arr); changed <- sum(arr!=original)
arr; before; after; changed
```

Expected Output / Analysis:

```
Sum before: 5
Final first layer:
     [,1] [,2] [,3]
[1,]    0    0    9
[2,]    5    8    0
Final second layer:
     [,1] [,2] [,3]
[1,]   40    0   10
[2,]    0   20    0
Sum after: 92; changed elements: 9
```

Result: Negative values are replaced by zero, then every value in layer 2 is multiplied by ten. The comparison counts elements whose final value differs from the original.



Topic 3: Classes in R – S3 Class, S4 Class and Reference Class

Question 1 (S3 · Level 3 – Apply)

Question focus: S3 student class and methods

Aim: Create classed student lists and dispatch print and summary methods.

Algorithm:

61. Identify the required R object and its dimensions or class.
62. Apply the relevant R function(s) or operators.
63. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

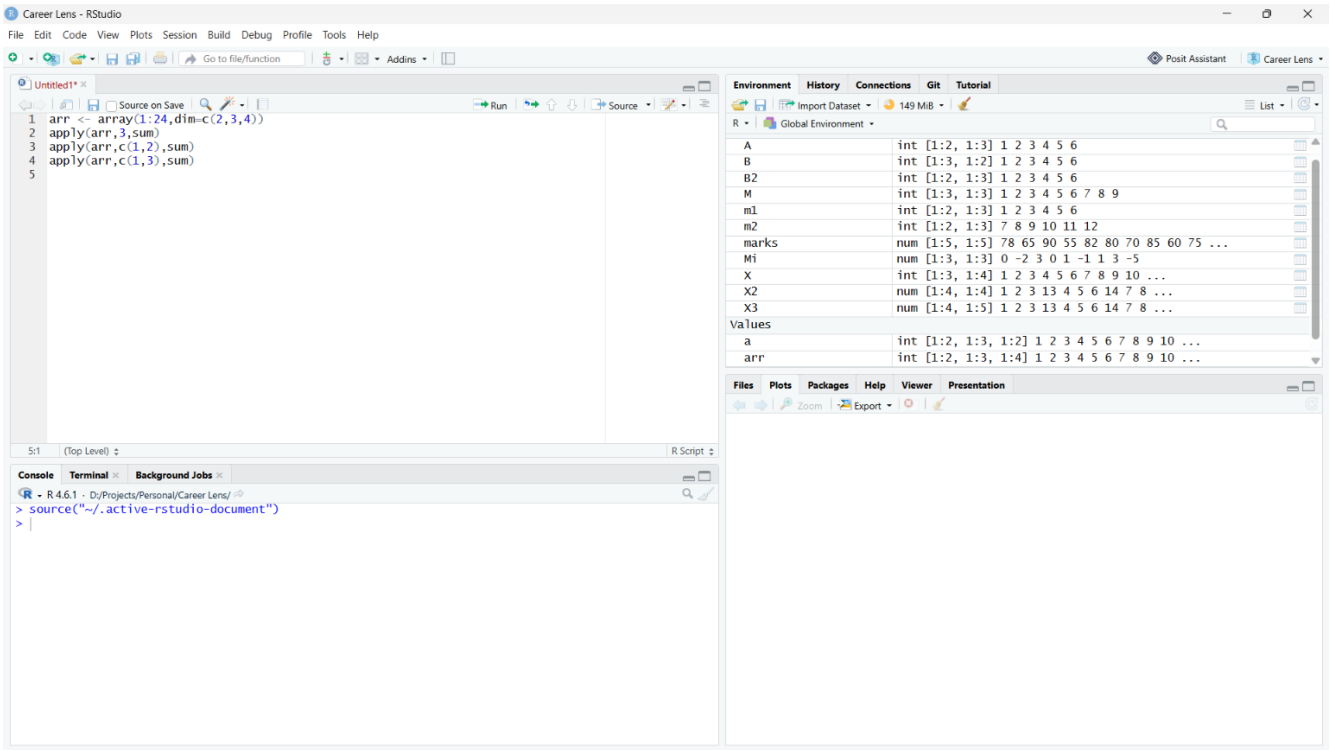
R Program:

```
new_student <- function(name,rollno,marks)
 structure(list(name=name,rollno=rollno,marks=marks),class="student")
print.student <- function(x,...)
cat("Name:",x$name,"\nRoll:",x$rollno,"\nMarks:",x$marks,"\n")
summary.student <- function(object,...) {
 grade <- if(object$marks>=90) "A" else if(object$marks>=75) "B" else
if(object$marks>=60) "C" else "D"
 c(name=object$name,grade=grade)
}
s1 <- new_student("Asha",101,92); s2 <- new_student("Ravi",102,68)
print(s1); print(s2); summary(s1); summary(s2)
```

Expected Output / Analysis:

```
Asha, roll 101, marks 92 -> grade A.
Ravi, roll 102, marks 68 -> grade C.
```

Result: print() and summary() are S3 generics; R selects print.student() and summary.student() from the object's class.



Question 2 (S3 · Level 4 – Analyze)

Question focus: Generic dispatch and NextMethod()

Aim: Trace methods for student, teacher, default and grad_student classes.

Algorithm:

64. Identify the required R object and its dimensions or class.
65. Apply the relevant R function(s) or operators.
66. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

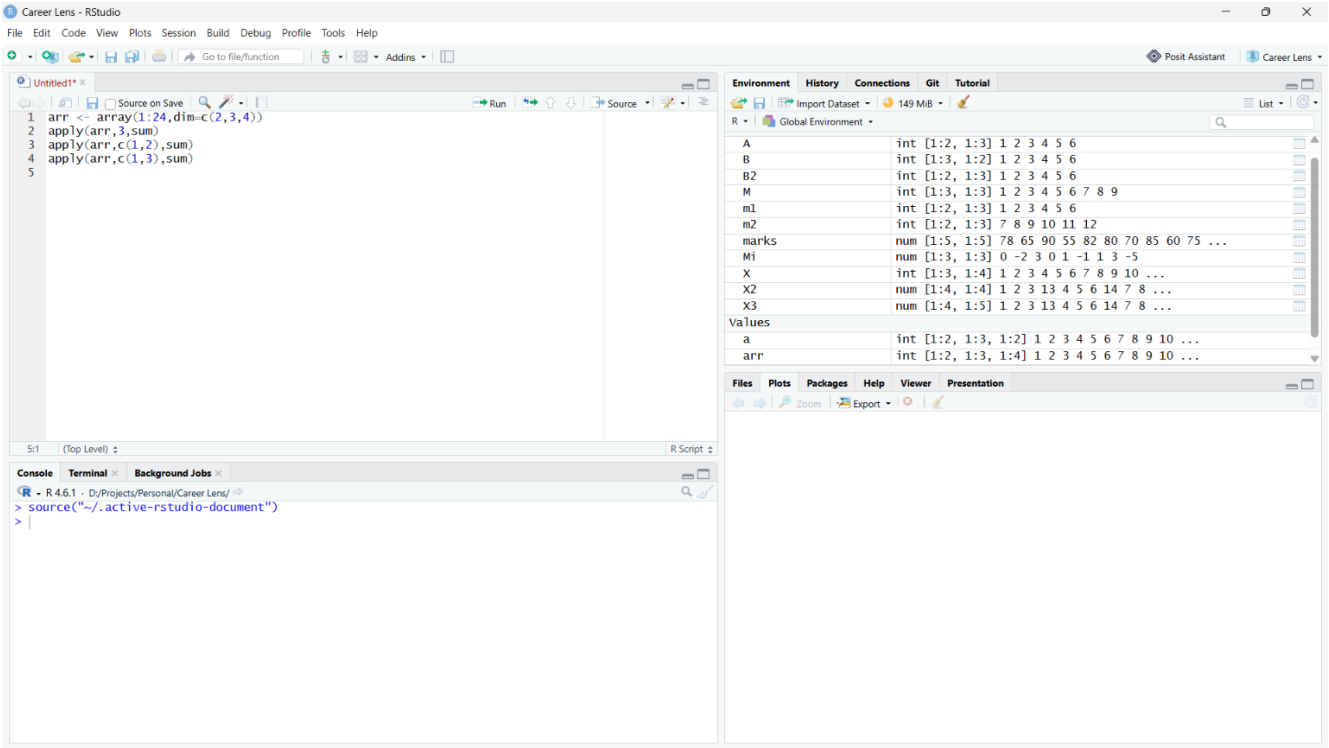
R Program:

```
info <- function(x) UseMethod("info")
info.student <- function(x) cat("student method\n")
info.teacher <- function(x) cat("teacher method\n")
info.default <- function(x) cat("default method\n")
info.grad_student <- function(x) {cat("grad_student method\n"); NextMethod()}
info(structure(list(),class="student"))
info(structure(list(),class="teacher"))
info(1:3)
info(structure(list(),class=c("grad_student","student")))
```

Expected Output / Analysis:

```
student method
teacher method
default method
grad_student method
student method
```

Result: UseMethod() searches the class vector. NextMethod() continues dispatch to the next applicable class.



Question 3 (S3 · Level 3 – Apply)

Question focus: Employee and manager pay methods

Aim: Create an employee base method and manager method with a 10% bonus.

Algorithm:

67. Identify the required R object and its dimensions or class.
68. Apply the relevant R function(s) or operators.
69. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

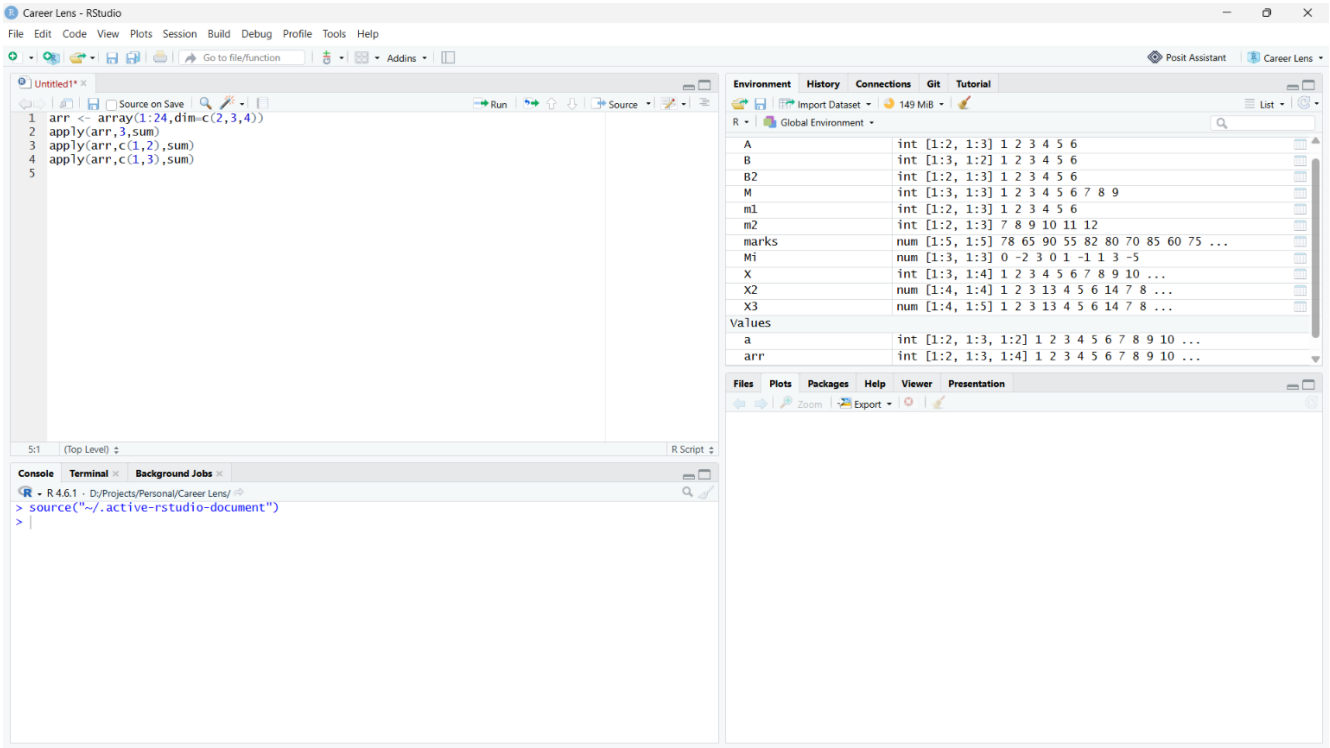
R Program:

```
pay <- function(x) UseMethod("pay")
pay.employee <- function(x) x$salary
pay.manager <- function(x) NextMethod()*1.10
e <- structure(list(name="Ravi",salary=50000),class="employee")
m <- structure(list(name="Asha",salary=70000),class=c("manager","employee"))
pay(e); pay(m)
```

Expected Output / Analysis:

```
Employee: 50000
Manager including 10% bonus: 77000
```

Result: The manager class is searched first. NextMethod() obtains the base salary from pay.employee(), then the manager method adds 10%.



Question 4 (S3 · Level 4 – Analyze)

Question focus: S3 validation versus S4 validation

Aim: Write a constructor and validator that enforce character name and numeric marks.

Algorithm:

70. Identify the required R object and its dimensions or class.
71. Apply the relevant R function(s) or operators.
72. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

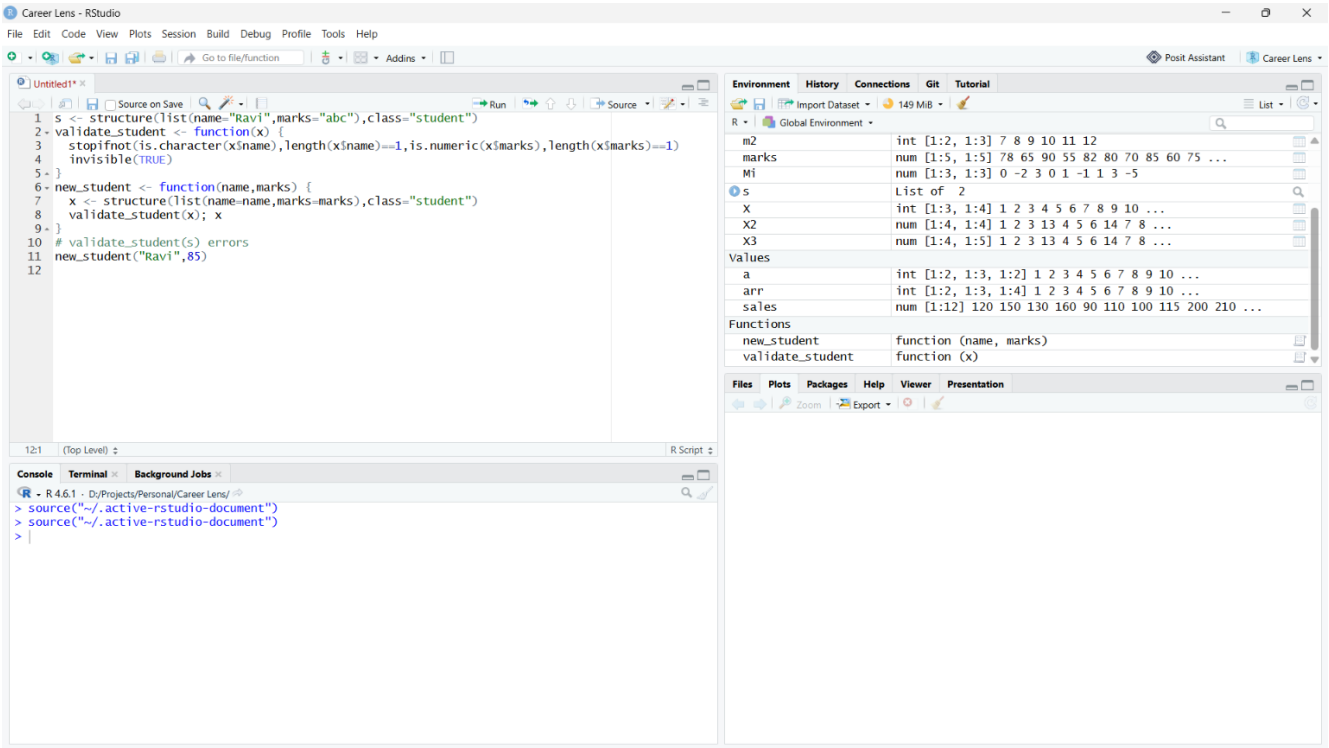
R Program:

```
s <- structure(list(name="Ravi",marks="abc"),class="student")
validate_student <- function(x) {
 stopifnot(is.character(x$name),length(x$name)==1,is.numeric(x$marks),length(x$marks)==1)
 invisible(TRUE)
}
new_student <- function(name,marks) {
 x <- structure(list(name=name,marks=marks),class="student")
 validate_student(x); x
}
# validate_student(s) errors
new_student("Ravi",85)
```

Expected Output / Analysis:

```
The classed list with marks="abc" is created, but validate_student(s) errors.
new_student("Ravi",85) succeeds.
```

Result: S3 does not enforce a typed slot schema. Validation is manual; S4 supports declared slot types and validity functions.



Question 5 (S4 · Level 3 – Apply)

Question focus: S4 Person class and greeting

Aim: Define typed slots, modify age, implement greet() and test class membership.

Algorithm:

73. Identify the required R object and its dimensions or class.
74. Apply the relevant R function(s) or operators.
75. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

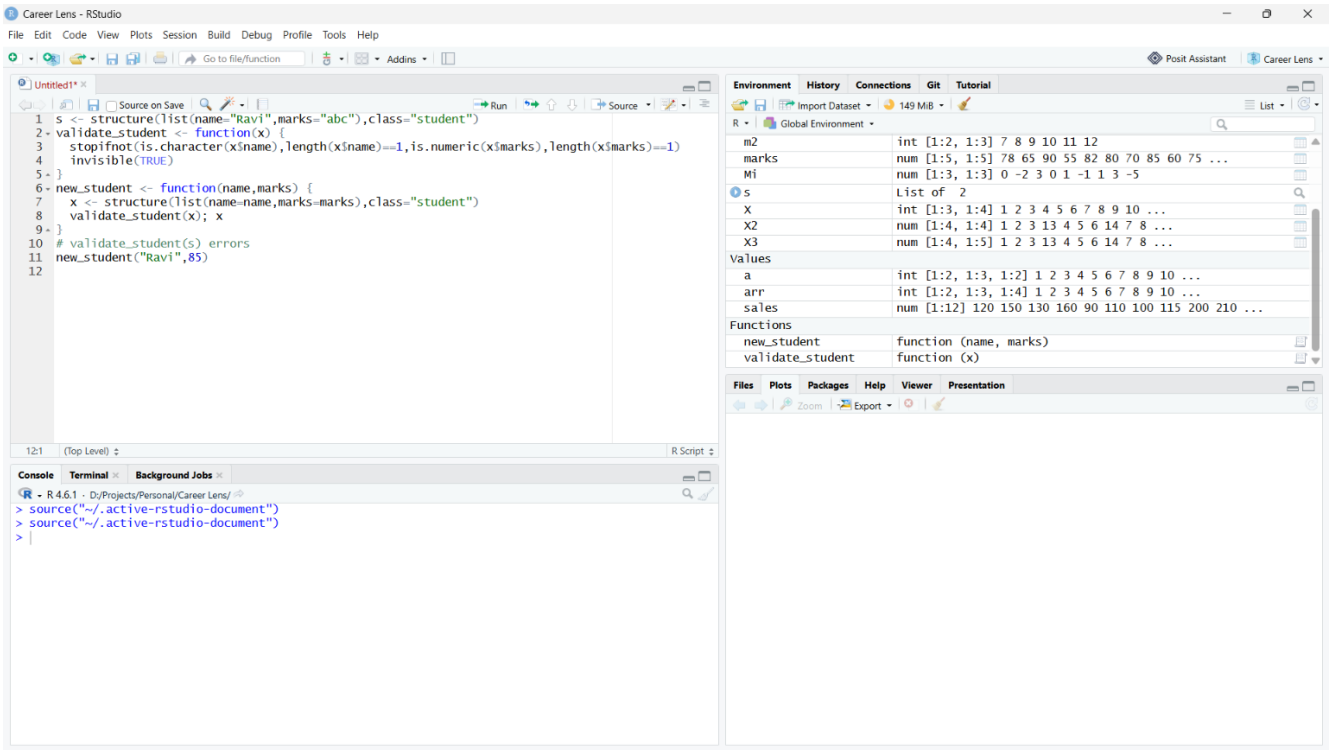
R Program:

```
setClass("Person",slots=c(name="character",age="numeric"))
p <- new("Person",name="Asha",age=20)
p@age <- 21
setGeneric("greet",function(object) standardGeneric("greet"))
setMethod("greet","Person",function(object) cat("Hello,",object@name,"\n"))
greet(p); is(p,"Person")
```

Expected Output / Analysis:

```
greet(p) prints Hello, Asha. is(p,"Person") returns TRUE; age is now 21.
```

Result: S4 uses formal slot declarations and setMethod() to register methods for generics.



Question 6 (S4 · Level 4 – Analyze)

Question focus: S4 validity and direct slot assignment

Aim: Reject negative age on construction and use validObject() after modification.

Algorithm:

76. Identify the required R object and its dimensions or class.
77. Apply the relevant R function(s) or operators.
78. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

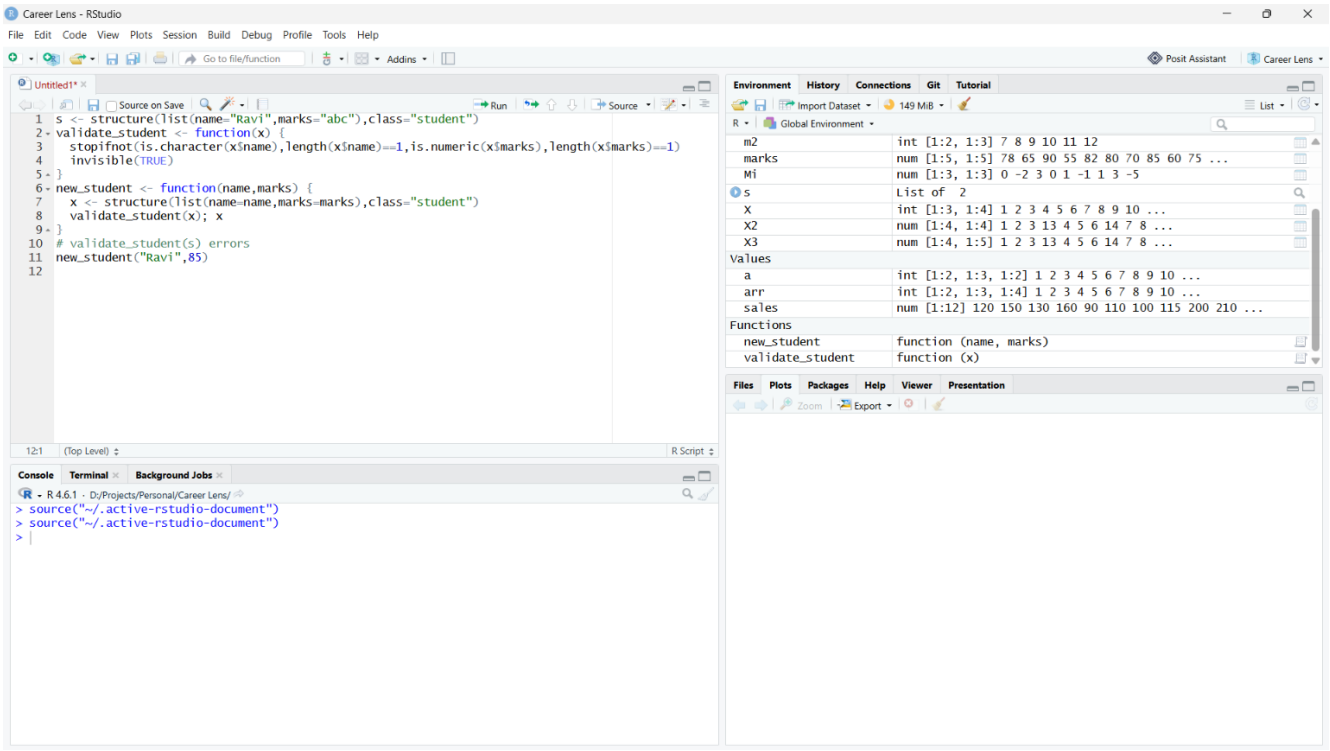
R Program:

```
setClass("Person",slots=c(name="character",age="numeric"),
 validity=function(object) {if(object@age<0) "age must be non-negative" else TRUE})
# new("Person",name="Asha",age=-5) fails
p <- new("Person",name="Asha",age=25)
p@age <- -10
validObject(p)
```

Expected Output / Analysis:

```
Creation with age=-5 fails validity. After p@age <- -10, validObject(p) reports "age must
be non-negative".
```

Result: S4 has formal slots and validity rules, but direct assignment may not immediately run the full validity function. Call validObject() when invariants must be checked.



Question 7 (S4 · Level 3 – Apply)

Question focus: S4 Student inheritance

Aim: Extend Person with marks and call the parent show method.

Algorithm:

79. Identify the required R object and its dimensions or class.
80. Apply the relevant R function(s) or operators.
81. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

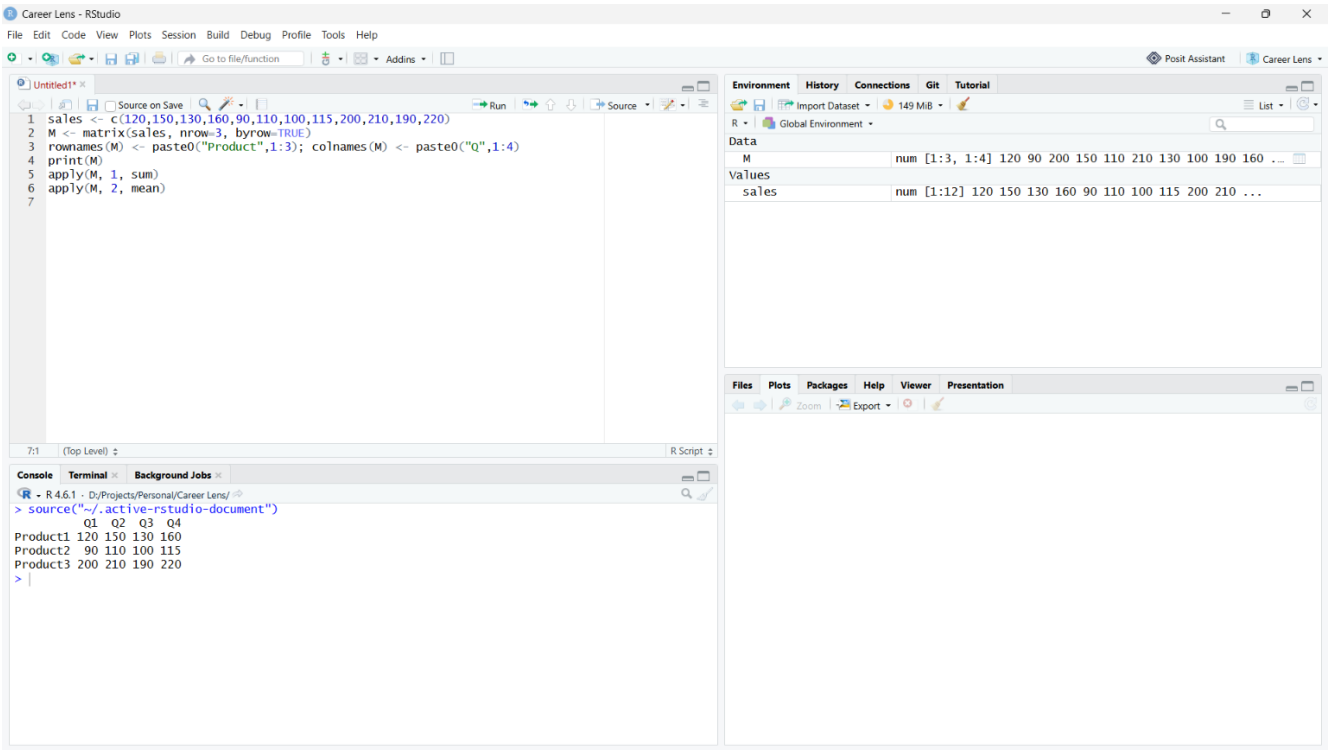
R Program:

```
setClass("Person",slots=c(name="character",age="numeric"))
setClass("Student",contains="Person",slots=c(marks="numeric"))
setMethod("show","Student",function(object) {callNextMethod();
cat("Marks:",object@marks,"\n")})
obj <- new("Student",name="Asha",age=20,marks=91)
show(obj); is(obj,"Person"); slotNames("Student")
```

Expected Output / Analysis:

```
Displays inherited Person details followed by Marks: 91. is(obj,"Person") is TRUE. Slots:
name, age, marks.
```

Result: Student inherits Person's slots and methods. callNextMethod() invokes the next applicable show method.



Question 8 (Reference Class · Level 3 – Apply)

Question focus: Mutable Account reference class

Aim: Implement deposit and withdraw with mutable balance and insufficient-funds feedback.

Algorithm:

82. Identify the required R object and its dimensions or class.
83. Apply the relevant R function(s) or operators.
84. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and
verify output → Stop.
```

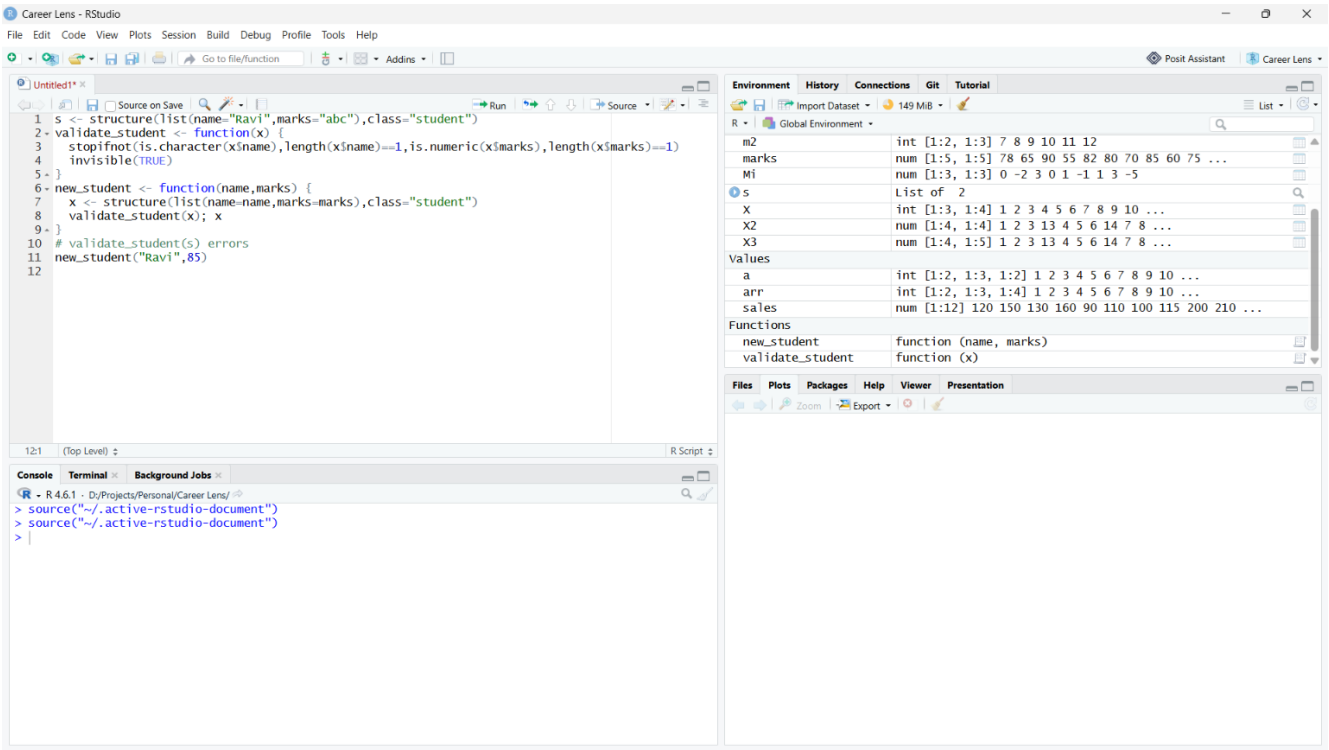
R Program:

```
Account <- setRefClass("Account",fields=list(balance="numeric"),methods=list(
 deposit=function(amount) {if(amount>0) balance <<- balance+amount;
cat("Balance:",balance,"\n")},
 withdraw=function(amount) {if(amount>balance) cat("Insufficient balance\n")
 else if(amount>0) balance <<- balance-amount
 cat("Balance:",balance,"\n")}
))
acc <- Account$new(balance=1000)
acc$deposit(500); acc$withdraw(2000); acc$withdraw(300); acc$balance
```

Expected Output / Analysis:

```
After deposit: 1500. Withdrawal of 2000 reports insufficient balance and leaves 1500.
Withdrawal of 300 leaves 1200.
```

Result: Reference class objects are mutable; <<- updates the object's balance field.



Question 9 (Reference Class · Level 4 – Analyze)

Question focus: Reference aliasing versus copy()

Aim: Predict balances after mutating a and compare b <- a with c <- a$copy().

Algorithm:

85. Identify the required R object and its dimensions or class.
86. Apply the relevant R function(s) or operators.
87. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

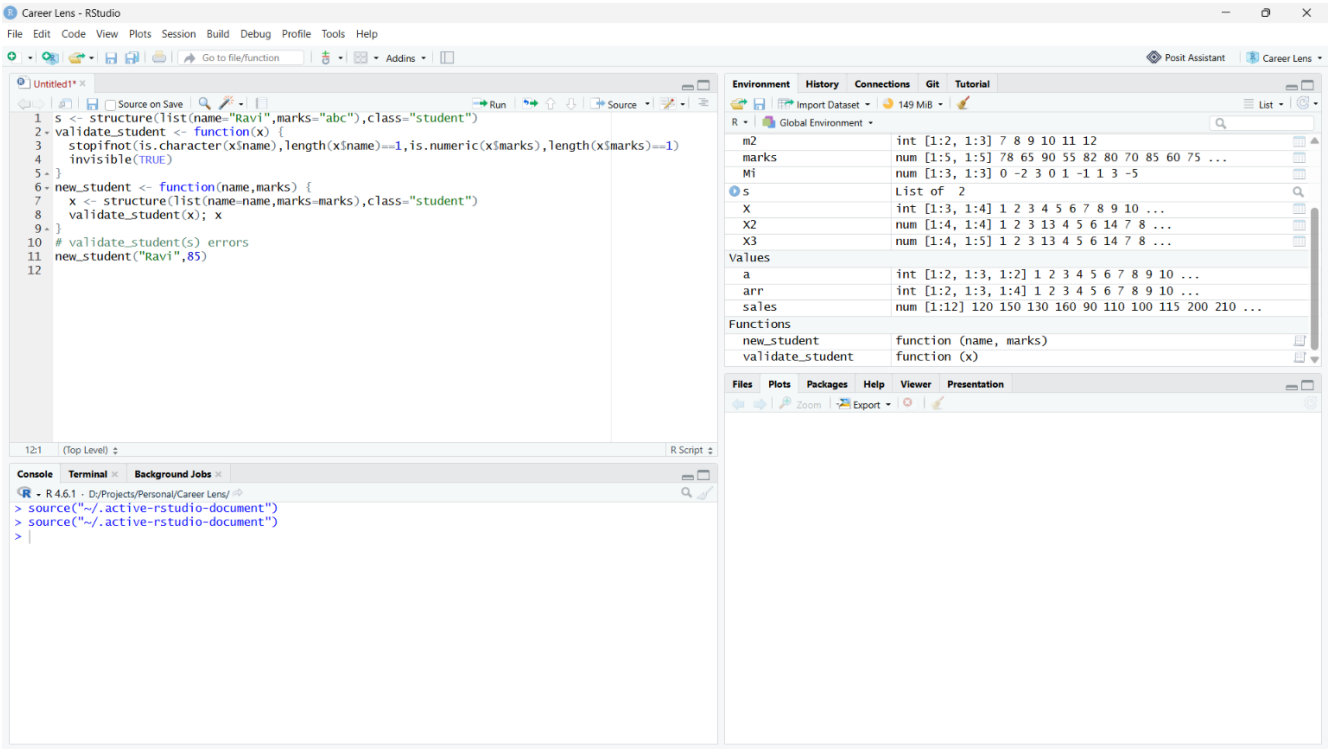
R Program:

```
Account <- setRefClass("Account",fields=list(balance="numeric"),
 methods=list(deposit=function(amount) balance <<- balance+amount))
a <- Account$new(balance=1000); b <- a; c <- a$copy()
a$deposit(500)
a$balance; b$balance; c$balance
```

Expected Output / Analysis:

```
a = 1500; b = 1500; c = 1000.
```

Result: a and b refer to the same object; c is a separate copy. S3/S4 objects generally follow copy-on-modify semantics rather than shared mutable reference behavior.



Question 10 (S3 vs S4 vs Reference · Level 4 – Analyze)

Question focus: Choose a class system for a bank application

Aim: Compare mutability, inheritance, method dispatch and validation for a bank application.

Algorithm:

88. Identify the required R object and its dimensions or class.

89. Apply the relevant R function(s) or operators.

90. Display the result and check it against the expected behaviour.

Pseudocode:

```
Start → Define the required data/object → Apply the specified operation(s) → Display and verify output → Stop.
```

R Program:

```
# S3: flexible, generic-function dispatch
x <- structure(list(balance=1000),class="account")
print.account <- function(x,...) cat(x$balance)
# S4: formal typed slots and inheritance
setClass("BankAccount",slots=c(balance="numeric"))
setClass("SavingsAccount",contains="BankAccount",slots=c(rate="numeric"))
# Reference Class: mutable state and object methods
BankRef <- setRefClass("BankRef",fields=list(balance="numeric"),
 methods=list(deposit=function(x) balance <<- balance+x))
```

Expected Output / Analysis:

```
S3 is flexible but lacks built-in typed slots. S4 provides formal slots, inheritance and validity. Reference Classes provide mutable fields and object-invoked methods.
```

Result: For state that changes over time, Reference Classes are a natural fit; S4 is strong where formal type constraints dominate. A drawback of Reference Classes is shared mutable state, which can complicate debugging.

